南京大學

研 究 生 毕 业 论 文
（申 请 硕 士 学 位）

论 文 题 目 面向广告与商业化内容的管理、应用与治理平台的设计与实现

作 者 姓 名 靳琦清

院 系 软件学院

专 业 名 称 软件工程

指 导 教 师 刘峰 高级工程师

2026 年 2 月 28 日

学　　号：**522024320076**

论文答辩日期：**xxxx** 年 **xx** 月 **xx** 日

指 导 教 师：　　（签字）

Design and Implementation of a Platform for the Management, Application, and Governance of Advertising and Commercial Content

by

QiQing Jin

Supervised by

Feng Liu, Senior Engineer

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of
MASTER
in
Software Engineering

Software Institute
Nanjing University

February 28, 2026

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向广告与商业化内容的管理、应用与治理平台的设计与实现

软件工程 专业 2024 级硕士生姓名：靳琦清

指导教师（姓名、职称）：刘峰 高级工程师

摘　要

随着互联网广告与商业化业务的快速发展，广告产品形态、业务规则及运营策略日益复杂，围绕广告投放、产品说明、业务规范与运营实践等内容不断沉淀，形成了规模庞大、类型多样的专业内容资产。这类内容在业务人员培训、运营决策支持以及客户服务等场景中发挥着重要作用。然而，随着业务线数量和内容规模的持续增长，现有内容平台在内容统一管理、高效应用以及质量保障等方面逐渐暴露出诸如内容分散、复用困难、维护成本高以及质量难以评估和治理等问题，已难以满足广告与商业化场景下的实际需求。

针对上述问题，本文在分析现有内容管理与知识平台相关工作的基础上，结合企业内部广告与商业化业务的实际应用场景，设计并实现了一套面向广告与商业化内容的管理、应用与治理平台。平台以内容全生命周期为核心，对内容的建模、组织、应用与治理进行系统化设计，旨在提升内容资产的统一管理能力、应用效果以及质量可控性。

在内容管理方面，平台提出了一种基于空间与应用分层的内容组织模型，实现对不同广告业务线及其应用场景的隔离与统一管理，并支持文章、问答、课程和案例等多种内容形态的规范化管理。结合类目树、标签体系、权限控制与审批流程，平台有效降低了内容维护成本，提升了内容资产的复用性与一致性。在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成（Retrieval-Augmented Generation，RAG）的智能问答能力，通过内容切片、向量索引、多通道召回与重排等技术手段，为上游智能应用提供高相关性的内容支持，拓展了内容资产的应用方式。在内容治理方面，针对内容规模扩大带来的质量问题，本文构建了一种基于重复度、歧义度与覆盖度的内容治理机制，结合向量检索与大语言模型实现对内容质量的自动检测，并通过线上化治理任务实现内容治理的闭环处理。此外，平台通过数据洞察模块对内

容使用效果和治理结果进行统计分析，为内容优化与治理策略调整提供数据支持。

实际应用结果表明，本文所设计和实现的平台在广告与商业化业务场景中有效提升了内容管理效率与应用效果，降低了内容治理的人力成本，对面向广告与商业化内容的内容平台建设具有一定的工程实践价值和参考意义。

关键词： 广告与商业化内容；统一内容模型；检索增强生成（RAG）；内容治理闭环；数据洞察

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Design and Implementation of a Platform for the Management, Application, and Governance of Advertising and Commercial Content

SPECIALIZATION: Software Engineering

POSTGRADUATE: QiQing Jin

MENTOR: Feng Liu, Senior Engineer

Abstract

With the rapid growth of Internet advertising and commercialization businesses, product forms, business rules, and operational strategies have become increasingly complex. A large volume of professional content has been accumulated around ad delivery, product documentation, business specifications, and operational practices, serving scenarios such as internal training, operational decision-making, and customer support. However, as both business lines and content scale up, existing content platforms increasingly suffer from fragmented content sources, limited reuse across scenarios, high maintenance costs, and insufficient capability for quality evaluation and governance.

To address these challenges, this thesis designs and implements a management, application, and governance platform for advertising and commercial content in an enterprise environment. Centered on the content lifecycle, the platform adopts a hierarchical organization model based on spaces and applications to achieve isolation and unified management across business lines, and supports multiple content types including articles, FAQs, courses, and cases. It further integrates taxonomy and tagging, fine-grained access control, and an approval-and-publishing workflow to improve consistency and reuse of content assets.

For content consumption, the platform provides both page-based browsing and search, and introduces retrieval-augmented generation (RAG) for intelligent question answering through content slicing, vector indexing, multi-channel retrieval, and reranking, enabling upstream intelligent applications to obtain highly relevant and traceable evidence. For content quality assurance, the platform establishes an automated governance mechanism focusing on duplication, ambiguity, and coverage, combining vector

retrieval with large language models and operationalizing the results as online governance tasks. In addition, a data insight module aggregates usage and governance metrics to support iterative optimization of content and governance strategies.

keywords: advertising and commercial content, unified content model, retrieval-augmented generation (RAG), content governance, data insight

目　　录

插图清单

附表清单

第一章　引言

1.1　项目背景

随着互联网广告与商业化业务的持续发展，广告产品形态、投放策略及业务规则不断演进，相关业务体系呈现出高度复杂化和专业化特征。在广告投放、产品能力说明、业务规范制定以及运营实践过程中，逐步沉淀了大量与广告与商业化业务密切相关的专业内容资产。这些内容涵盖业务规则解读、产品功能说明、操作流程指导以及典型业务案例等，是支撑广告与商业化业务运行和决策的重要基础。

随着业务规模扩大和业务线数量增加，广告与商业化内容在数量和类型上呈现持续增长趋势。内容逐步分散于不同业务线和系统中，缺乏统一的组织模型和管理规范，导致内容检索效率降低、复用成本上升，内容维护高度依赖人工经验，难以形成系统化、可持续的内容管理体系。

在内容规模不断扩大的同时，内容的应用方式也在持续演进。传统依赖人工检索和页面浏览的内容使用模式，已难以满足广告与商业化业务对效率和准确性的要求。随着检索增强生成和智能问答等技术在业务系统中的逐步应用，内容的使用方式正从“人工查找”向“智能获取”转变。这一变化对内容的结构化程度、可检索性以及一致性提出了更高要求，同时也显著提升了内容在业务决策和问题处理过程中的使用频率。

随着内容被更广泛、更高频地应用，内容质量问题逐渐凸显。一方面，部分内容在长期演进过程中形成重复和冗余，降低了内容体系的整体可用性；另一方面，不同内容在描述同一业务问题时可能存在表述不一致或语义歧义，在智能问答等自动化应用场景下容易被进一步放大，影响内容理解的准确性。此外，由于缺乏系统化分析和评估手段，内容体系在部分业务领域存在覆盖不足的问题。随着内容规模和应用深度的提升，传统依赖人工维护的方式已难以有效支撑内容质量的持续保障。

现有内容管理平台多以内容存储和页面展示为核心，主要提供基础的内容管理功能，在内容统一建模、多形态内容协同管理、智能化应用支持以及内容

质量治理等方面能力有限，难以满足广告与商业化场景下内容规模大、应用频繁且质量要求高的实际需求。基于上述背景，本文设计并实现了一套面向广告与商业化内容的管理、应用与治理平台，从内容全生命周期视角出发，对内容的生产、组织、应用和维护过程进行了系统性设计。平台在统一内容建模与组织方式的基础上，引入智能化的内容应用能力以及自动化与人工相结合的内容治理机制，实现了广告与商业化内容资产的统一管理、高效应用与质量可控，为复杂广告与商业化业务场景下的内容使用与持续维护提供了一种工程化解决方案。

1.2 国内外发展现状及分析

在内容管理与知识应用领域，国内外研究和工程实践主要围绕企业内容管理系统（Enterprise Content Management，ECM）和内容管理系统（Content Management System，CMS）展开。相关研究普遍关注内容的集中存储、统一建模、流程化管理以及权限控制等问题，通过引入内容生命周期管理和工作流机制，提高内容管理的规范性与可控性。在政务、教育及企业信息化场景中，此类系统已形成较为成熟的工程实践体系，在文档协同、权限隔离和合规管理方面发挥了重要作用。

在工程实践层面，国外以 Microsoft 推出的 SharePoint、Atlassian 的 Confluence 等平台为代表，提供了面向组织级内容的统一管理与协作能力，支持内容版本控制、权限配置以及结构化组织。这类平台在内容管理和协同办公方面具备较高成熟度，但其核心使用模式仍以人工检索和页面浏览为主，对内容的深度应用和智能化利用支持相对有限。国内企业也普遍建设了面向内部知识沉淀的内容管理或知识平台，用于支撑业务培训和经验复用，但整体上仍以管理和展示为主要目标。

随着业务复杂度和内容规模的持续提升，传统内容管理系统在内容应用层面的局限逐渐显现。近年来，随着大语言模型技术的发展，基于检索增强生成（Retrieval-Augmented Generation，RAG）的内容应用与智能问答技术成为研究热点。相关研究通过将外部知识检索与生成模型相结合，在一定程度上提升了知识密集型任务的准确性，并围绕检索策略、结果重排以及生成效果评估等方面展开了大量探索。部分研究进一步将 RAG 技术应用于多模态文档和垂直领域场景，引入图片、表格等非文本内容的检索与理解能力，用于提升复杂业务

问题的问答效果。

尽管 RAG 等智能内容应用技术在算法层面取得了显著进展，但现有研究多聚焦于模型能力和问答效果优化，通常假设知识来源具备较高质量和良好结构，较少关注内容在生产、维护和演进过程中的管理问题。在实际业务场景中，内容往往来源多样、形态复杂且质量参差不齐，单纯依赖智能检索和生成模型，难以从根本上保障内容应用的稳定性和可靠性。

与此同时，在内容质量与数据治理方向，已有研究从一致性、完整性、准确性以及治理流程等维度提出了较为系统的理论框架，强调通过指标化评估、流程化管理和工具化支撑提升内容与数据资产的可信度和可维护性。这类研究在数据治理和企业信息管理领域具有较成熟的理论基础。然而，在具体工程实践中，内容质量治理往往以离线分析或人工审查为主，与内容管理系统和智能应用系统相对独立，缺乏将质量检测结果直接反馈到内容管理和应用流程中的闭环机制，难以支撑内容规模持续增长和高频应用场景下的长期运营需求。

综合来看，现有研究和系统在内容管理、智能应用以及内容治理等方面均取得了一定成果，但整体上仍呈现出相对割裂的发展状态。传统内容管理系统侧重内容管理但缺乏智能化应用能力，智能问答和 RAG 相关研究强调内容应用效果却忽视内容管理和治理的工程复杂性，而内容质量治理研究又难以与实际内容应用形成闭环。在面向广告与商业化内容的复杂业务场景中，内容规模大、更新频繁且应用方式多样，单一侧重某一方面的解决方案难以满足实际需求。因此，如何在统一内容管理的基础上，引入智能化内容应用能力，并通过系统化的内容治理机制保障内容质量，实现管理、应用与治理协同演进的内容平台，仍有待进一步研究和工程实践。

1.3 本文主要工作

广告与商业化内容在实际业务中具有规模大、更新频繁、内容形态多样以及应用场景复杂等特点，传统内容平台在统一管理、高效应用和质量保障等方面逐渐暴露出不足。为解决上述问题，本文在分析国内外内容管理、智能内容应用以及内容治理相关研究和工程实践的基础上，结合企业内部广告与商业化业务的实际需求，设计并实现了一套面向广告与商业化内容的管理、应用与治理平台，对平台的整体架构与核心模块进行了系统性设计与实现。

本文所设计的平台以内容全生命周期管理为核心，对内容从创建、组织、

应用到治理的全过程进行统一支撑，减少内容在不同系统之间的割裂状态，提高内容资产的复用效率和质量可控性。围绕上述目标，本文的主要工作包括以下几个方面。

首先，在内容管理方面，设计并实现了一套统一的内容管理机制。平台通过引入空间与应用的分层组织模型，对不同广告业务线及其应用场景进行隔离与管理，支持文章、问答、课程和案例等多种内容形态的统一建模与维护。同时，结合类目树、标签体系、权限控制和审批流程，实现内容从创建、维护到上线的规范化管理，降低内容维护成本，提高内容资产的一致性和可复用性。

其次，在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成的智能问答能力，拓展内容资产的使用方式。针对不同内容形态的特点，设计了相应的内容切片、索引和召回策略，并通过多通道召回与重排机制提升内容匹配的相关性，为上游智能应用提供稳定、可靠的内容支撑，使内容能够在复杂业务场景中被高效获取和利用。

再次，在内容治理方面，针对内容规模扩大和应用频率提升带来的质量问题，构建了一套内容治理机制。平台从内容重复度、歧义度和覆盖度等维度对内容质量进行分析，通过向量检索和大语言模型判别实现内容质量的自动检测，并将检测结果转化为线上化治理任务，指派相关人员进行处理，从而形成内容治理的闭环流程，提升内容体系的整体质量和可维护性。

最后，在数据分析与反馈方面，平台通过数据洞察模块对内容的使用情况和治理效果进行统计分析，从页面访问、用户会话和内容使用等多个维度对内容应用效果进行量化评估，为内容优化和治理策略调整提供数据支持，进一步支撑平台的持续演进和优化。

综上所述，本文通过对广告与商业化内容管理、应用与治理平台的设计与实现，探索了一种将内容管理、智能应用与内容治理进行一体化整合的工程化解决方案，为复杂业务场景下内容资产的高效利用和长期维护提供了实践参考。

1.4 论文的组织结构

本文围绕面向广告与商业化内容的管理、应用与治理平台的设计与实现展开，全文共分为六章，各章节内容安排如下。

第一章为绪论，主要介绍了论文的研究背景和研究意义，分析了广告与商

业化内容在实际业务场景下面临的管理、应用和质量治理问题，并对国内外相关研究和工程实践的发展现状进行了综述，最后概述了本文的主要研究内容和工作安排。

第二章为相关技术综述，主要介绍了本文系统设计与实现过程中所涉及的关键技术和方法，包括内容管理相关技术、全文检索与向量检索技术、检索增强生成（RAG）技术以及大语言模型在智能问答中的应用等，为后续系统设计与实现提供技术基础。

第三章为平台需求分析与总体设计。该章节结合广告与商业化业务场景，对平台的功能需求和非功能需求进行了分析，在此基础上给出了平台的总体架构设计，并对各核心模块的设计思路进行了说明。

第四章为平台的详细设计与实现。该章节围绕内容管理、内容应用和内容治理等核心模块，对系统的关键功能设计和具体实现过程进行了详细阐述，并结合实际系统界面展示了平台的主要功能效果。

第五章为系统效果分析与评估。该章节通过对平台在实际业务场景中的运行情况进行分析，从内容管理效率、内容应用效果以及内容治理成效等多个维度对系统进行评估，验证平台设计的可行性和有效性。

第六章为总结与展望，对全文的研究工作进行了总结，分析了当前系统存在的不足，并对未来在功能扩展和技术优化等方面的研究方向进行了展望。

第二章　相关技术综述

2.1　远程过程调用 RPC

远程过程调用（Remote Procedure Call，RPC）是一种用于支持分布式系统中服务间通信的技术机制，其核心思想是通过统一的调用抽象，使得应用程序能够以类似本地函数调用的方式访问远程服务，从而屏蔽底层网络通信细节。通过 RPC，开发者无需直接处理消息传输、序列化以及连接管理等复杂问题，即可实现跨进程、跨节点的服务调用，这在现代分布式系统和微服务架构中具有广泛应用。

在 RPC 通信模型中，调用方通过接口定义与被调用方建立逻辑关联，调用请求在客户端侧被封装为消息并通过网络发送至服务端，服务端在完成对应逻辑处理后将结果返回给调用方。整个过程在调用语义上通常表现为同步调用，使得业务逻辑能够以顺序化的方式进行组织。通过这种方式，RPC 技术在保持调用语义清晰性的同时，实现了服务之间的解耦和功能拆分。

从系统实现角度看，RPC 框架通常围绕接口描述、数据序列化、通信协议以及调用调度等关键环节进行设计。接口描述用于定义服务的调用规范，序列化机制负责将结构化数据转换为可传输的二进制格式，通信协议则用于保障请求与响应在网络中的可靠传输。通过对这些环节进行统一封装，RPC 框架能够在不同语言和运行环境之间提供一致的调用体验。

在分布式系统中，RPC 技术还常与服务发现、负载均衡和容错机制相结合，以提升系统的可用性和扩展性。通过动态路由和调用调度，RPC 框架可以在多个服务实例之间分发请求，在部分节点发生故障时自动进行切换，从而保障服务调用的连续性。这种能力使 RPC 成为构建高可用分布式服务体系的重要基础。

总体而言，RPC 技术通过将网络通信抽象为过程调用，为分布式系统提供了一种高效、清晰的服务交互方式。其在接口化设计、通信封装以及调用治理等方面的工程实践，为现代服务化架构和复杂业务系统的构建提供了重要支撑。

2.2 文档数据库 MongoDB

MongoDB 是一种面向文档的 NoSQL 数据库，采用灵活的文档数据模型，广泛应用于内容管理、日志存储以及高并发读写等业务场景。MongoDB 以 BSON（Binary JSON）格式存储数据，能够自然映射应用层对象结构，支持对结构化与半结构化数据的统一存储，具有模式灵活、扩展性强等特点，在互联网和企业级系统中得到广泛应用。

在数据组织方式上，MongoDB 以数据库（Database）和集合（Collection）作为数据管理的基本单位，每条数据以文档形式存储在集合中。与传统关系型数据库相比，MongoDB 不强制要求固定的表结构，不同文档可以包含不同字段，从而能够灵活适应业务数据结构的演进需求。这种文档式存储方式在内容频繁变化或字段不固定的场景下具有较高的适用性。

在查询与索引机制方面，MongoDB 提供了丰富的查询接口，支持基于字段的精确查询、范围查询以及条件组合查询等操作。系统支持为文档字段建立多种类型的索引，包括单字段索引、复合索引和全文索引等，以提升查询性能。同时，MongoDB 提供聚合管道机制，能够对集合中的数据进行过滤、分组和统计分析，满足一定程度的数据分析需求。

在系统架构层面，MongoDB 支持副本集（Replica Set）和分片（Sharding）机制。副本集通过主从复制提高数据的可用性和容错能力，当主节点发生故障时能够自动完成主节点切换；分片机制则通过将数据分布到多个节点上，实现数据的水平扩展，支持大规模数据存储和高并发访问场景。

在分布式应用中，MongoDB 常用于承载对结构灵活性要求较高的数据存储需求，适合存储内容元数据、非结构化信息以及业务过程数据。其灵活的数据模型和良好的扩展能力，使其能够在业务规模增长和数据结构演进过程中保持较高的适应性，为复杂系统的数据管理提供稳定支撑。

2.3 消息队列 RocketMQ

Apache RocketMQ 是一种高性能、高可靠的分布式消息中间件，最初由阿里巴巴提出并开源，现已成为 Apache 顶级项目，广泛应用于异步通信、系统解耦和流量控制等业务场景。RocketMQ 采用发布—订阅（Publish-Subscribe）的消息模型，支持点对点和广播等多种消费方式，能够满足大规模分布式系统

中对消息传递可靠性和稳定性的要求。

在消息组织方式上，RocketMQ 以主题（Topic）作为消息管理的基本单位，消息在 Broker 中按照队列（Queue）进行存储和分发。通过将同一主题下的消息分布到多个队列中，系统可以实现消息的并行写入和并行消费，从而提升整体吞吐能力。在消息存储方面，RocketMQ 采用顺序写磁盘的方式进行消息持久化，在保证可靠性的同时提高了写入效率。

在消息投递与消费机制方面，RocketMQ 支持生产者向指定主题发送消息，由消费者以订阅方式进行消费，并通过消费进度管理和投递确认机制保障消息的可达性。系统支持至少一次（At-Least-Once）的消息投递语义，并提供顺序消息和延迟消息等能力，使其能够适应对消息时序性和定时触发有要求的业务场景。

在系统架构层面，RocketMQ 由 NameServer、Broker、Producer 和 Consumer 等组件构成。NameServer 负责维护集群的路由信息，Broker 负责消息的存储与转发，Producer 和 Consumer 分别承担消息的生产与消费职责。该架构采用去中心化设计，支持 Broker 节点的动态扩展和故障转移，在节点异常情况下能够保证消息服务的连续性。

在分布式系统中，RocketMQ 常用于构建基于事件驱动的消息通信机制，通过消息队列对请求进行缓冲和调度，降低系统之间的直接依赖关系。同时，消息队列能够在高并发场景下对突发流量进行削峰填谷，并通过消费速率控制实现限速处理，从而提升整体系统的稳定性和可扩展性。

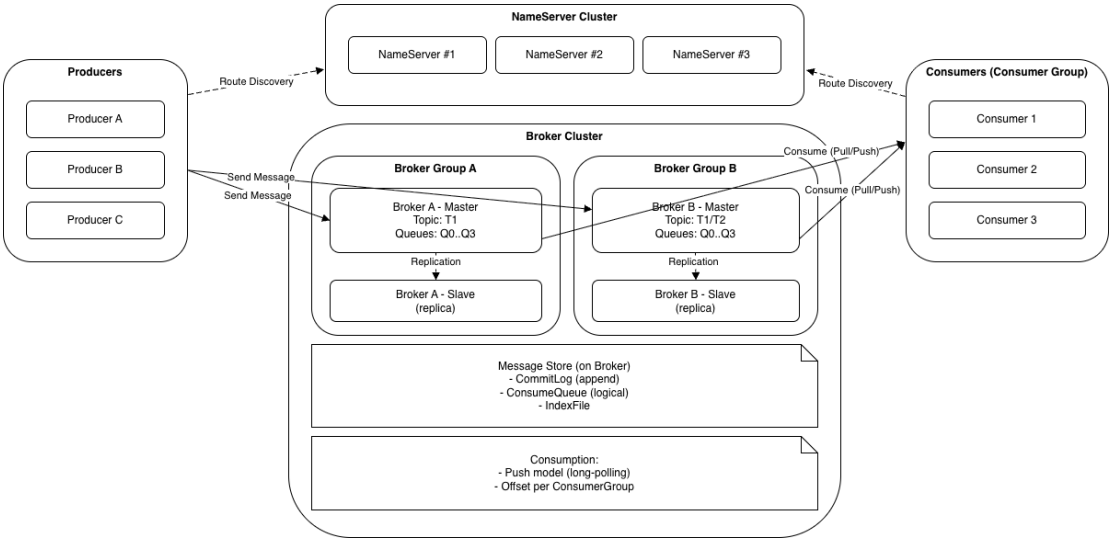

图 2-1: RocketMQ 架构图

2.4 全文搜索引擎 ElasticSearch

ElasticSearch 是一种基于 Lucene 构建的分布式搜索与分析引擎，广泛应用于全文检索、日志分析和实时数据查询等场景。该系统采用面向文档的数据模型，支持对结构化、半结构化以及非结构化数据的统一存储与检索，具有高并发、低延迟和良好可扩展性等特点，已成为内容管理与检索系统中的重要基础组件。

在数据组织方式上，ElasticSearch 以索引作为数据管理的基本单位，每个索引由多个分片组成，分片是实际存储和查询的最小单元。通过将分片分布到不同节点上，系统能够实现查询请求的并行处理，从而在数据规模较大时仍保持较高的查询性能。在文本检索方面，ElasticSearch 基于倒排索引结构对文档内容进行索引，将词项与文档之间建立映射关系，使系统能够高效支持关键词匹配、短语查询以及相关性计算等全文检索功能。

在查询机制方面，ElasticSearch 提供了基于 DSL 的查询方式，支持精确匹配、全文匹配、范围查询以及多条件组合查询等多种检索模式。在全文检索场景下，系统通常采用 BM25 等相关性算法对文档进行评分，并按照相关性得分对查询结果进行排序，以优先返回与查询条件匹配度较高的内容。同时，ElasticSearch 支持在查询过程中结合过滤条件和聚合分析，实现对内容数据的统计与分析，满足复杂业务场景下的检索与分析需求。

在系统架构层面，ElasticSearch 通过主分片与副本分片机制保障数据的高可用性，当部分节点发生故障时，系统能够自动完成分片迁移和主从切换，保证服务的连续性。此外，ElasticSearch 支持节点的动态扩缩容，能够根据业务数据规模和访问压力灵活调整集群规模，具备良好的横向扩展能力，适合内容规模持续增长的业务场景。

在内容管理与应用场景中，ElasticSearch 非常适合用于支撑基于关键词和结构化条件的内容检索需求。一方面，其全文检索能力能够支持用户快速定位相关内容；另一方面，通过对内容类型、标签、类目等字段建立索引并进行组合查询，可以实现内容筛选、聚合展示等功能。在本文所设计的平台中，ElasticSearch 主要用于承载内容的全文检索与结构化检索能力，与向量检索等技术形成互补，为内容的页面化展示和智能化应用提供高效、稳定的基础检索支撑。

2.5 Embedding 与向量检索技术

Embedding 技术是一种将文本、图片等非结构化数据映射为高维向量表示的方法，其核心目标是将语义信息编码到向量空间中，使语义相近的内容在向量空间中距离更近。通过 Embedding，原本难以直接进行数学计算的自然语言或多模态内容可以被转换为统一的数值表示形式，从而为后续的相似度计算和检索提供基础。在近年来的自然语言处理和信息检索领域中，Embedding 已成为语义理解和语义匹配任务中的关键技术。

在内容表示层面，Embedding 模型通常基于深度学习方法，对文本中的上下文语义进行建模，将词语、句子或文档映射为固定维度的向量。与传统基于关键词的表示方式相比，Embedding 能够在一定程度上捕捉语义相似性，即使不同文本在表面词汇上存在差异，其向量表示仍可能较为接近。这一特性使得 Embedding 在处理同义表达、语义扩展等场景中具有明显优势，适合用于复杂内容语义建模。

在 Embedding 的基础上，向量检索技术用于解决在大规模向量集合中高效查找相似向量的问题。向量检索的基本思想是，通过定义向量之间的相似度度量方式，对查询向量与候选向量进行比较，从而返回语义上最为接近的结果。在实际应用中，常用的相似度度量方式包括余弦相似度和向量内积等，这些方法能够较好地反映向量之间的语义接近程度。

随着内容规模的不断增长，简单的全量向量比对在性能上难以满足实际需求，因此向量检索系统通常通过构建专门的索引结构来提升查询效率。通过对向量空间进行组织和近似搜索，系统可以在保证一定召回效果的前提下显著降低检索开销，从而支持大规模数据场景下的实时相似度查询。这类索引化的向量检索机制已成为现代语义搜索系统中的重要组成部分。

在内容管理与知识应用场景中，Embedding 与向量检索技术能够为内容提供基于语义的检索能力，使系统不再局限于关键词匹配，而是能够从语义层面发现相关内容。通过将内容表示为向量并引入向量检索机制，系统可以在内容规模较大、表达方式多样的情况下，仍然实现较高质量的相关内容召回，为后续的智能应用奠定基础。

2.6 检索增强生成 RAG

检索增强生成（Retrieval-Augmented Generation，RAG）是一种将信息检索与生成模型相结合的应用范式，旨在提升生成模型在知识密集型任务中的准确性和可靠性。与仅依赖模型参数进行生成的方式不同，RAG 在生成阶段引入外部知识检索过程，使生成模型能够基于检索到的相关内容进行回答，从而在一定程度上缓解大语言模型在知识更新滞后和事实幻觉等方面的局限。

RAG 技术的提出源于大语言模型在实际应用中的局限性。尽管预训练模型在语言理解和生成能力上取得了显著进展，但其内部知识主要来源于训练阶段的静态语料，难以覆盖最新或特定领域的知识，同时在面对知识密集型问题时容易生成与事实不一致的内容。为解决上述问题，研究者开始探索将外部知识源引入生成过程的技术路径。在这一背景下，Patrick Lewis 等人在 2020 年提出了 RAG 框架，该工作首次系统性地将信息检索模块与生成模型进行结合，通过在生成过程中引入外部文档作为条件输入，显著提升了知识密集型任务的回答效果，为后续相关研究奠定了基础。

在 RAG 的基本流程中，系统通常首先根据用户输入执行检索操作，从外部知识库中召回与问题语义相关的内容；随后，将检索结果与用户输入一并作为上下文信息提供给生成模型，由模型综合多条检索信息生成最终回答。通过这种“先检索、后生成”的方式，RAG 将外部知识的可控性与生成模型的表达能力结合起来，使生成结果不再完全依赖模型参数中隐含的知识。

随着语义表示和向量检索技术的发展，RAG 架构逐步从基于关键词的检索方式演进为基于语义的检索方式。通过引入 Embedding 和向量检索机制，系统能够在更大规模和更复杂的知识库中高效获取语义相关内容，提高检索结果的相关性和覆盖度。这一阶段的研究重点逐渐从“是否引入检索”转向“如何提升检索质量与生成效果之间的协同能力”，推动 RAG 从概念性方法演进为具有工程可行性的系统范式。

在工程实践中，RAG 技术已被广泛应用于知识问答、智能客服和文档理解等场景。相较于传统基于检索的问答系统，RAG 不再直接返回检索结果，而是通过生成模型对多条信息进行综合理解和组织，生成更加连贯和自然的回答；相较于纯生成式模型，RAG 通过引入外部知识来源，提高了生成结果的可解释性和可控性，使系统能够更好地适应知识更新频繁和领域知识复杂的应用环境。

总体而言，RAG 作为一种将信息检索与生成模型相结合的系统级应用范式，为智能内容应用提供了有效的技术路径。通过在生成过程中引入外部知识检索机制，RAG 在准确性、灵活性和可扩展性之间取得了较好的平衡，已成为当前知识密集型应用和企业级智能问答系统中的重要技术方向。

2.7 本章小结

本章主要介绍了远程过程调用 RPC、文档数据库 MongoDB、消息队列 RocketMQ、全文搜索引擎 ElasticSearch、Embedding 与向量检索技术以及检索增强生成（RAG）技术的基本原理和应用场景。其中，RPC 技术是一种用于分布式系统中实现跨进程或跨节点服务通信的技术机制；MongoDB 是一种基于文档模型的 NoSQL 数据库，适用于存储结构灵活、模式多变的数据；RocketMQ 是一种高性能的分布式消息中间件，常用于异步通信、系统解耦和流量控制；ElasticSearch 是一种分布式全文搜索与分析引擎，能够支持大规模内容的高效检索与统计分析；Embedding 与向量检索技术通过语义表示和相似度计算，实现了基于语义的内容召回能力；RAG 技术则通过将信息检索与生成模型相结合，为知识密集型应用提供了更为准确和可控的生成式能力。这些技术在内容管理、内容检索以及智能应用等方面发挥着重要作用，为本文后续系统的需求分析、架构设计与功能实现提供了必要的技术支撑。

第三章　内容管理、应用与治理平台的分析与设计

3.1　系统整体概述

面向广告与商业化内容的管理、应用与治理平台是一个以内容资产全生命周期为核心的综合性平台系统，目标是在内容规模持续增长、业务线与应用场景不断扩展的背景下，实现内容的统一建模、规范化管理、高效应用以及质量可控。平台以“内容资产化”和“能力平台化”为设计导向，将内容从分散的业务沉淀与人工维护过程，转化为可组织、可复用、可追踪、可治理的系统化资产，并通过统一接口对上游业务系统、智能助手与运营工具提供稳定服务。

从业务视角看，平台服务于广告与商业化生态中的多类使用场景，包括业务人员的知识获取与运营决策支持、面向外部的产品说明与规范指引、以及客户服务和智能问答等高频咨询场景。上述场景对内容提出了共同要求：一是内容来源多、形态多样，且跨业务线复用需求强；二是内容更新频繁，依赖人工维护易产生不一致与过期；三是内容消费方式从“页面浏览”逐步演进到“搜索与问答驱动”，要求更高的检索质量与可解释性；四是内容质量问题需要可度量、可触发、可闭环，否则难以支撑规模化运营。因此，本平台在总体设计上采用“管理与应用一体、治理闭环驱动”的思路，以内容组织与权限体系为基础，以检索与智能问答能力为牵引，以质量治理和数据洞察为反馈闭环，实现内容体系的持续演进。

从系统视角看，平台整体由内容管理、内容应用、内容治理与数据洞察四个核心域组成，并通过统一的数据模型、权限体系与事件机制实现协同。内容管理域负责内容的生产与维护，包括空间与应用的分层组织、内容形态管理（文章、问答、课程、案例等）、权限控制与审批发布等能力。该域的核心作用是将多来源、多形态内容纳入统一规范，形成可控的内容资产底座，并为后续检索、问答与治理提供一致的数据输入和业务语义。

内容应用域面向内容消费与智能化使用需求，提供两类能力：其一是面向

页面化场景的内容展示与检索能力，通过统一内容接口支撑列表、目录导航、详情页、关键词搜索、标签聚合等常见页面组件；其二是面向智能问答场景的检索增强生成能力，通过内容切片、索引构建、多通道召回与重排等手段，为上游智能应用提供高相关性、可追溯的内容依据。与传统“内容库 + 搜索”不同，平台在应用域强调“可控检索”和“结果可解释”，即召回结果需可定位到具体内容片段与来源，以便在治理与迭代中形成明确反馈路径。

内容治理域面向内容规模化带来的质量问题，构建“自动检测—任务化分发—人工处理—结果回收”的闭环机制。治理能力覆盖重复度、歧义度与覆盖度等典型问题：重复度治理用于识别内容冗余与重复维护；歧义度治理用于检测同一问题下的内容冲突、不一致或表述歧义；覆盖度治理用于从知识体系与用户需求侧识别内容缺口与薄弱点。平台将治理结果以标准化的治理任务形式落地，包括问题定位、影响范围、建议处理方式与责任归属，配合任务状态流转，使治理过程可追踪、可度量、可持续运营，从而避免治理仅停留在离线分析或一次性专项。

数据洞察域承担平台运行效果与治理成效的量化评估职责，通过对页面访问、用户会话、检索与问答链路日志、内容使用情况等数据进行统计分析，输出面向运营与产品迭代的指标体系。该域既用于评价内容应用效果（如内容被召回次数、生成引用次数、转人工率等），也用于反哺治理策略（如高频问题聚类、低命中内容识别、质量问题分布等），从而形成“应用驱动生产、数据反哺治理”的闭环运营机制。

在关键流程层面，平台围绕内容全生命周期形成若干核心链路：在内容生产链路中，内容经创建与编辑后进入审批与发布流程，已发布状态内容作为对外可用的内容资产进入索引与应用链路；在索引构建链路中，平台对不同内容形态进行标准化处理（如切片、结构化字段抽取、元数据归一）并构建全文索引与向量索引，以支撑检索与问答；在内容消费链路中，用户通过页面浏览、搜索或智能问答触发内容召回，平台返回内容或为生成模型提供依据；在质量治理链路中，平台周期性或事件驱动地对内容进行质量检测，将发现的问题沉淀为治理任务并推动处理，处理结果再反向影响内容库与索引；在评估反馈链路中，数据洞察持续沉淀关键指标与行为数据，为内容生产策略、检索策略与治理规则提供优化依据。

总体而言，本平台以统一内容资产底座为核心，将内容管理、内容应用、内容治理与数据洞察四个域有机整合：内容管理域负责内容的生产与规范化，

为后续应用与治理提供一致的数据输入；内容应用域面向不同消费场景提供展示、检索与问答能力，实现内容价值的直接输出；内容治理域通过质量检测与任务化闭环，保障内容体系的持续演进；数据洞察域则通过效果评估与问题定位，形成“应用驱动生产、数据反哺治理”的反馈机制。四个域相互协同、闭环迭代，既覆盖传统内容平台的基础管理需求，也面向智能化内容消费方式提供支撑，为广告与商业化业务在知识沉淀、信息一致性以及智能化服务能力方面提供稳定、可扩展的工程化方案。

3.2 内容管理、应用与治理平台需求分析

3.2.1 用户角色

本平台的用户角色如表**??**所示，主要分为四种，分别是：平台负责人、业务负责人、业务运营和客户。平台负责人负责整个平台层面的配置与运维管理；业务负责人负责某一业务空间下内容体系的整体管理与维护；业务运营是平台内容生产和日常运营的主要执行角色；客户主要作为内容的消费方，通过页面或智能服务获取平台提供的内容支持。

表 3-1: 用户角色表

用户角色	描述
平台负责人	平台的最高管理角色，负责整个平台的基础配置、空间管理及全局权限控制，保障平台稳定运行。
业务负责人	负责某一业务线或业务空间下内容体系的管理与运维，对内容结构、应用配置及业务权限进行整体把控。
业务运营	负责平台内容的创建、编辑和维护等日常运营工作，并在权限范围内查看相关内容使用数据。
客户	平台内容的消费角色，包括广告主、代理商等，通过页面浏览、搜索或智能助手等方式使用平台内容。

除上述用户角色外，平台还需要与上游智能应用（如智能助手）进行接口对接。该类调用方通常以系统身份访问平台的内容检索与问答能力，其行为数据将被纳入会话分析与效果评估范围。

3.2.2 内容管理功能需求

内容管理功能的用例图如图 **??**所示。内容管理模块作为平台的核心模块，主要负责内容资产的全生命周期管理，包括内容的生产、维护、组织、权限控制以及发布上线等环节。该模块旨在将分散在不同业务线和系统中的内容进行统一纳管，通过标准化的模型和流程，确保内容在流转过程中的一致性、安全性和可控性。

内容管理用例图占位（待补充）

图 3–1: 内容管理用例图

内容管理模块的功能需求可进一步拆分为多个相互衔接的功能块，以覆盖内容从组织边界建立、内容生产维护到权限约束与发布上线的全链路。各功能块共同构成内容资产的可管、可控与可追溯能力。为保证论述结构清晰，本节按照功能块依次给出需求说明，并为每一类子需求列举 1–3 个代表性用例。

（1）空间与应用管理。平台需要支持以空间为单位建立业务隔离与治理边界。空间用于承载内容资产的业务归属、成员范围与默认权限策略，并支持空间的创建、编辑以及基础配置维护。为适配不同消费场景，平台需在空间内支持应用配置：应用定义内容对外呈现的组织方式与可用能力，包括可用内容形态、类目树与标签体系、检索范围与排序规则等。空间与应用管理功能主要包含创建空间与创建应用等用例。其中，创建空间与创建应用用例描述分别如表 **??**与表 **??**所示。

表 3-2: 创建空间用例描述表

ID	CM-05
用例名称	创建空间
参与者	平台负责人
用例描述	创建业务空间以作为内容资产的组织边界与权限边界，并初始化空间基础配置。
触发条件	新业务线接入或现有业务需要建立独立治理边界。
前置条件	1. 当前用户具备空间管理权限。
后置条件	1. 系统生成空间 ID 并创建默认成员组与默认权限策略。
基本流程	1. 用户进入空间管理页面并点击新建空间。2. 系统弹出空间信息编辑弹窗。3. 用户填写空间名称、业务归属等信息并保存。4. 系统校验信息并持久化保存空间配置。
替代流程	1. 若空间名称重复或信息校验失败，系统提示原因并要求用户修改。

表 3-3: 创建应用用例描述表

ID	CM-06
用例名称	创建应用
参与者	业务负责人
用例描述	在指定空间内创建应用并配置内容组织方式，以适配不同内容消费场景。
触发条件	业务需要新增内容门户、帮助中心或面向智能应用的知识库入口。
前置条件	1. 目标空间已存在且当前用户具备应用管理权限。
后置条件	1. 系统生成应用 ID 并初始化应用配置（可用形态、类目树与标签配置等）。
基本流程	1. 用户进入空间详情页并点击创建应用。2. 用户选择应用类型并填写应用名称与基础配置。3. 系统校验配置并创建应用。4. 系统返回应用创建成功提示并展示应用配置入口。
替代流程	1. 若应用配置不完整（如未配置可用内容形态），系统提示并阻止创建。

（2）多形态内容服务。平台需要支持多种内容形态的统一纳管与一致化维护。不同内容形态在结构与消费方式上存在差异，但应具备统一的基础管理能力，包括创建、编辑与保存草稿等。多形态内容服务的典型用例包括创建内容、编辑内容与删除内容，其用例描述分别如表 **??**、表 **??**与表 **??**所示。

表 3–4: 创建内容用例描述表

ID	CM-01
用例名称	创建内容
参与者	业务运营
用例描述	在指定空间与应用下创建新的内容实体，支持 knowledge、faq、course、case 等多种形态，并生成初始草稿。
触发条件	业务需要新增产品说明、规则解读或运营素材等内容资产。
前置条件	1. 目标空间与应用已创建并处于可用状态。2. 当前用户拥有该空间下的内容创建权限。
后置条件	1. 系统生成唯一的内容 ID。2. 内容状态置为“草稿”。
基本流程	1. 用户进入内容管理列表页，点击“新建”按钮。2. 用户选择内容形态（如文章、问答等）。3. 系统初始化空白草稿并跳转至编辑器。4. 用户填写标题、正文及必要的属性字段（如类目、标签）。5. 用户点击“保存”按钮。6. 系统校验必填项与格式，校验通过后持久化存储。
替代流程	1. 若校验失败，系统提示具体错误字段，用户修改后可重新保存。

表 3-5: 编辑内容用例描述表

ID	CM-02
用例名称	编辑内容
参与者	业务运营
用例描述	对已存在的内容进行修改，包括正文更新与属性调整，并保存修改结果以进入后续提审与发布流程。
触发条件	内容信息发生变更或需要优化现有内容。
前置条件	1. 内容处于可编辑状态（如草稿、驳回或已发布）。2. 当前用户拥有该内容的编辑权限。
后置条件	1. 内容变更被持久化保存。
基本流程	1. 用户在内容列表点击目标内容的“编辑”按钮。2. 系统加载内容数据并进入编辑器。3. 用户修改正文或属性信息。4. 用户点击“保存”或“提交”。5. 系统保存修改并记录变更。
替代流程	1. 若校验失败，系统提示需要补全或修正的字段。

表 3-6: 删除内容用例描述表

ID	CM-03
用例名称	删除内容
参与者	业务运营、业务负责人
用例描述	将不再使用的内容从应用侧移除或标记删除，使其不再对外可见。
触发条件	内容过期、废弃或创建错误。
前置条件	1. 当前用户拥有删除权限。
后置条件	1. 内容在管理端被移除或标记删除，并在应用侧不可见。
基本流程	1. 用户选中一条或多条内容，点击“删除”按钮。2. 系统弹出二次确认弹窗，提示删除后果。3. 用户确认删除。4. 系统执行删除操作并更新内容状态。
替代流程	1. 若内容已被其他应用引用，系统提示“存在引用关系”，阻止删除或提示解除引用。

（3）权限管理。平台需要构建基于角色的访问控制（RBAC）体系，在空间与应用维度实现权限隔离与精细化授权。权限粒度应覆盖模块级与操作级，

既能控制用户进入某一空间/应用的访问权限，也能控制内容的创建、编辑、删除、提审、审批、上线与下线等具体操作权限。权限管理功能主要包含成员角色分配与权限策略配置等用例，其用例描述如表 **??**与表 **??**所示。

表 3–7: 成员角色分配用例描述表

ID	CM-07
用例名称	成员角色分配
参与者	平台负责人，业务负责人
用例描述	为目标用户在空间或应用维度分配角色，以确定其可访问范围与可执行操作集合。
触发条件	新成员加入空间/应用或成员职责变更需要调整权限。
前置条件	1. 目标空间与应用已存在。2. 当前用户具备成员管理权限。
后置条件	1. 系统更新用户角色映射并立即生效。
基本流程	1. 管理员进入空间或应用的成员管理页面。2. 管理员选择目标用户并设置角色。3. 系统校验管理权限与角色合法性。4. 系统保存变更并返回成功提示。
替代流程	1. 若当前用户无权限执行授权操作，系统拒绝并提示原因。

表 3–8: 权限策略配置用例描述表

ID	CM-08
用例名称	权限策略配置
参与者	平台负责人
用例描述	配置空间或应用的默认权限策略与操作权限集合，实现对模块与操作级权限的精细化控制。
触发条件	业务治理要求调整默认权限口径或新增权限项。
前置条件	1. 当前用户具备全局权限配置能力。
后置条件	1. 系统更新权限策略并对后续鉴权生效。
基本流程	1. 用户进入权限配置页面选择目标空间/应用。2. 用户配置角色与权限项映射关系并保存。3. 系统校验配置合法性并持久化保存。
替代流程	1. 若配置导致关键角色缺失必要权限，系统提示风险并阻止保存。

（4）审核与发布。内容对外可用的前提是准确性与合规性可控。平台需要提供流程化的审核与发布机制，对内容状态流转进行约束。内容在生命周期中至少应包含草稿、待审批、已发布、已下线等状态，状态变更需满足权限校验与流程条件。审核与发布功能主要包含提交审核、内容审批与内容上下线等用例，其用例描述分别如表 **??**、表 **??**与表 **??**所示。

表 3–9: 提交审核用例描述表

ID	CM-09
用例名称	提交审核
参与者	业务运营
用例描述	将内容草稿提交进入审批流程，申请将内容上线并对外可用。
触发条件	草稿内容完成编辑，需要上线以支撑页面展示、检索或智能问答。
前置条件	1. 内容处于可提交状态且基础校验通过。2. 当前用户具备提审权限。
后置条件	1. 系统创建审批记录并将内容状态更新为待审批。2. 通知审批者。
基本流程	1. 用户在内容详情页点击提交审核。2. 系统执行内容完整性校验与权限校验。3. 系统创建审批单并将内容状态置为待审批。4. 系统向用户返回提审成功提示。
替代流程	1. 若权限校验失败，系统拒绝提交并提示原因。2. 若内容校验失败，系统提示需要补全或修正的字段。

表 3-10: 内容审批用例描述表

ID	CM-10
用例名称	内容审批
参与者	业务负责人
用例描述	对待审批内容进行审核并给出通过或拒绝结论，决定内容能否上线并对外可见。
触发条件	审批者在审批列表中发现待处理任务或收到待审批通知。
前置条件	1. 内容状态为待审批。2. 当前用户具备审批权限。
后置条件	1. 若审批通过，系统将内容状态更新为已发布，并发布内容变更事件供下游消费。2. 若审批拒绝，系统将内容状态更新为未通过并记录拒绝原因。
基本流程	1. 审批者进入审批列表并打开目标审批单。2. 审批者查看内容关键信息与变更点。3. 审批者选择通过或拒绝并提交意见。4. 系统更新审批单状态并回写内容状态。
替代流程	1. 若审批者选择拒绝，系统要求填写拒绝原因并通知发布者。

表 3-11: 内容上下线用例描述表

ID	CM-04
用例名称	内容上下线
参与者	业务负责人、业务运营
用例描述	控制内容在应用端的可见性。上线即发布内容使其对外可见，下线即撤回发布内容。
触发条件	内容审核通过需发布，或内容失效需紧急下线。
前置条件	1. 上线前需通过审批流程。2. 下线需具备相应管理权限。
后置条件	1. 更新内容发布状态（已发布/已下线）。2. 触发索引更新事件，同步搜索与问答端数据。
基本流程	1. 用户对“待发布”内容点击“上线”，或对“已发布”内容点击“下线”。2. 系统校验当前状态与权限。3. 系统更新内容状态并发布更新事件到 MQ。
替代流程	1. 若状态校验失败或权限不足，系统拒绝操作并提示原因。

（5）其他功能。除核心链路外，平台还需提供辅助能力以提升内容组织效率与复用效果，包括类目管理、标签管理以及应用引用机制等。其他功能主要包含类目管理、标签管理与应用引用内容等用例，其用例描述分别如表 **??**、表 **??**与表 **??**所示。

表 3-12: 类目管理用例描述表

ID	CM-11
用例名称	类目管理
参与者	业务负责人，业务运营
用例描述	在应用维度维护类目树结构，用于构建内容导航与主题聚合入口。
触发条件	新增业务主题或需要调整内容导航结构。
前置条件	1. 目标应用已存在且当前用户具备类目管理权限。
后置条件	1. 系统更新类目树配置。
基本流程	1. 用户进入应用配置中的类目管理页面。2. 用户新增、编辑或调整类目节点层级。3. 系统校验类目结构合法性并保存。
替代流程	1. 若调整导致类目形成循环或层级不合法，系统提示并阻止保存。

表 3-13: 标签管理用例描述表

ID	CM-12
用例名称	标签管理
参与者	业务运营
用例描述	维护标签体系并将标签应用于内容，以支持跨类目的聚合检索与运营配置。
触发条件	需要新增运营标签、调整标签口径或清理废弃标签。
前置条件	1. 目标应用已存在且当前用户具备标签管理权限。
后置条件	1. 系统更新标签集合并记录变更历史。
基本流程	1. 用户进入标签管理页面。2. 用户新增、编辑或禁用标签。3. 系统校验标签名称与冲突并保存。
替代流程	1. 若标签名称重复或校验失败，系统提示原因并要求用户修改。

表 3-14: 应用引用内容用例描述表

ID	CM-13
用例名称	应用引用内容
参与者	业务负责人，业务运营
用例描述	在应用维度引用其他应用或其他空间下的已发布内容，使其在本应用中可见与可用，并记录引用关系以支撑复用与影响范围分析。
触发条件	需要将权威口径内容在多个应用复用，避免重复维护与口径不一致。
前置条件	1. 被引用内容已发布且未下线。2. 当前用户具备应用配置权限并对被引用内容具备查看权限。
后置条件	1. 系统记录应用-内容引用关系并使引用在应用侧生效。
基本流程	1. 用户进入应用配置页面选择“内容引用”。2. 用户选择引用来源（空间/应用）并检索目标内容。3. 用户确认引用并保存配置。4. 系统校验权限与内容状态后创建引用关系并返回成功提示。
替代流程	1. 若内容未发布或权限不足，系统拒绝引用并提示原因。

3.2.3 内容应用功能需求

内容应用功能的用例图如图**??**所示。内容应用模块的主要功能是对内容进行展示、检索与问答，以满足不同场景下的内容消费需求。一方面，平台需要支持面向门户与管理后台的页面化展示与检索能力，帮助用户通过目录导航、列表筛选与关键词搜索快速定位内容；另一方面，平台需要面向智能问答等上游智能应用提供检索增强生成能力，使用户能够以对话方式获取结构化且可追溯的内容依据。

内容应用用例图占位（待补充）

图 3-2: 内容应用用例图

内容应用模块的核心用例可归纳为以下几类。

（1）内容展示。面向页面化消费场景，平台需提供内容目录与列表能力，支持按空间、应用、类目与标签等维度进行内容聚合展示，并提供详情页渲染与内容关联跳转能力，保障内容在不同业务线与不同应用中可被稳定复用。

（2）内容搜索。平台需提供关键词搜索与结构化筛选能力，支持对标题、正文、标签与类目等字段的组合查询，并保证检索结果在权限范围内可见。为提升体验，搜索结果需支持高亮、排序与分页等能力。

（3）智能问答。平台需提供面向智能问答场景的内容支撑能力：当用户以自然语言提问时，系统能够在指定空间与应用范围内检索相关内容并生成回答，同时返回可追溯的引用来源，以保障结果可解释并降低口径不一致风险。

本节选取三个关键用例进行描述。

表 3-15: 内容展示用例描述表

ID	CA-01
用例名称	内容展示
参与者	客户，业务运营
用例描述	用户在内容门户或管理后台按目录导航与列表聚合浏览内容，并查看内容详情。
触发条件	用户希望通过类目、标签或列表入口快速定位并阅读内容。
前置条件	1. 目标应用存在已发布内容。2. 用户具备对应空间与应用的访问权限。
后置条件	系统展示内容列表与详情，并记录访问行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台。2. 用户通过类目树或标签进入内容列表。3. 用户点击某一条内容进入详情页查看正文与关联信息。
替代流程	1. 若内容无权限访问或已下线，系统提示并返回可访问的替代内容或上级列表。

表 3-16: 内容检索用例描述表

ID	CA-02
用例名称	内容检索
参与者	客户，业务运营
用例描述	用户通过关键词与筛选条件检索内容，系统返回权限范围内的相关结果列表。
触发条件	用户需要定位某一产品说明、业务规则或典型案例。
前置条件	1. 目标空间与应用存在已发布内容。2. 用户具备对应空间与应用的访问权限。
后置条件	系统返回结果列表并记录检索行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台的搜索页面。2. 用户输入关键词并选择筛选条件。3. 系统执行权限过滤与检索召回。4. 系统按相关性与业务规则排序并返回结果。
替代流程	1. 若无匹配结果，系统提示并引导用户调整关键词或筛选条件。

表 3-17: 生成式问答用例描述表

ID	CA-03
用例名称	生成式问答
参与者	客户
用例描述	用户以自然语言提问，系统结合召回证据生成回答并返回引用来源。
触发条件	用户希望通过对话快速获取业务规则解释或操作指导。
前置条件	1. 用户具备问答入口的访问权限。2. 系统可获取足够的召回证据或具备降级策略。
后置条件	系统返回回答与引用，并记录会话明细与反馈信息用于评估。
基本流程	1. 用户在智能助手中输入问题。2. 系统完成内容召回与证据组织。3. 生成模型基于证据生成回答。4. 系统返回回答与引用来源。
替代流程	1. 若问题不在知识覆盖范围内，系统提示并引导用户转人工或查看相关内容列表。

3.2.4 内容治理功能需求

内容治理功能的用例图如图**??**所示。内容管理模块为平台提供了统一的内容资产底座，内容应用模块实现了内容的展示与智能问答，然而随着内容规模扩大与消费频率提升，重复冗余、表述冲突与覆盖不足等质量问题会直接影响检索效果与问答准确性。因此，内容治理模块的主要功能是对内容质量进行指标化检测，并以任务形式推动治理闭环，与前述模块协同形成“生产—应用—治理—反馈”的完整链路，保证内容体系在持续演进过程中的可维护性与可控性。

内容治理用例图占位（待补充）

图 3-3: 内容治理用例图

内容治理模块的核心用例包括治理检测、治理任务创建与分发、治理处理与审核、治理结果回收与追踪等。平台将治理问题抽象为标准化任务，任务中应包含问题类型、问题定位（内容与片段）、影响范围、建议处理方式与责任归属，从而使治理过程可追踪、可统计。

（1）重复度检测。识别内容体系中的重复条目与冗余片段，支持对重复内容进行聚合展示与合并建议，减少重复维护带来的成本与口径不一致风险。

（2）歧义度检测。针对同一问题或同一概念在不同内容中的表述冲突与语义歧义进行检测，给出冲突片段与候选一致口径，降低智能问答场景下的误导概率。

（3）覆盖度检测。结合用户查询与会话数据识别内容缺口，定位高频问题无有效内容支撑的领域，并形成补充内容的治理建议。

本节选取三个关键用例进行描述。

表 3-18: 质量检测用例描述表

ID	CG-01
用例名称	质量检测
参与者	平台负责人
用例描述	系统对指定范围内内容执行重复度、歧义度或覆盖度检测，输出可定位的问题集合与治理建议。
触发条件	需要对内容体系的质量问题进行阶段性治理或例行巡检。
前置条件	1. 指定空间与应用范围可用。2. 已具备相似度计算、冲突判定或缺口识别能力。
后置条件	系统生成检测结果并可转化为治理任务。
基本流程	1. 用户在治理模块选择检测范围与检测类型。2. 系统执行对应检测并生成问题定位与影响范围。3. 系统输出问题集合与建议处理方式。
替代流程	1. 若检测范围过大导致资源不足，系统支持分批检测并输出进度。

表 3-19: 治理任务创建用例描述表

ID	CG-02
用例名称	治理任务创建
参与者	平台负责人
用例描述	将检测结果转化为可分发的治理任务，明确责任人与处理要求。
触发条件	检测结果需要进入治理闭环并推动相关人员处理。
前置条件	已产生可定位的质量问题集合。
后置条件	1. 系统创建治理任务并进入待处理状态。2. 系统通知责任人。
基本流程	1. 用户在检测结果中选择问题条目并点击创建任务。2. 用户设置任务类型、优先级与责任人。3. 系统创建任务并生成问题定位与建议处理内容。
替代流程	1. 若同一问题已存在未完成任务，系统提示合并或追加处理。

表 3-20: 治理任务处理用例描述表

ID	CG-03
用例名称	治理任务处理
参与者	业务运营
用例描述	责任人根据任务定位对内容进行合并、修订、补充或下线，并提交处理结果。
触发条件	责任人收到治理任务并需要完成内容修订。
前置条件	1. 责任人具备对应内容的编辑权限。2. 任务处于可处理状态。
后置条件	1. 内容变更进入相应发布流程。2. 任务状态更新并记录处理结果。
基本流程	1. 责任人打开治理任务查看问题定位与建议。2. 责任人跳转至对应内容进行修订或合并。3. 责任人提交任务处理说明并完成。4. 系统更新任务状态并触发后续校验或复检。
替代流程	1. 若处理涉及多内容协同，系统支持拆分子任务并分别跟踪。

3.2.5 数据洞察功能需求

数据洞察功能的用例图如图**??**所示。内容管理模块负责内容的生产与发布，内容应用模块实现内容的消费与问答，内容治理模块推动质量问题的检测与闭环处理，而数据洞察模块则作为平台的“反馈中枢”，对上述模块的运行效果进行量化评估，支撑运营决策与平台迭代。该模块通过对页面访问、检索与问答链路、用户会话等数据进行采集与分析，输出面向运营与治理的指标体系，使平台能够形成“应用驱动生产、数据反哺治理”的闭环。

数据洞察用例图占位（待补充）

图 3–4: 数据洞察用例图

数据洞察模块的关键用例包括页面数据监控、会话维度分析、内容维度分析与会话明细。平台通过对页面访问、检索与问答链路日志进行采集与聚合，支撑运营人员从页面、会话与内容三个视角对平台效果进行量化评估，并通过会话明细下钻实现问题定位，为内容生产与治理策略调整提供依据。

表 3-21: 页面数据监控用例描述表

ID	DI-01
用例名称	页面数据监控
参与者	业务负责人，业务运营
用例描述	用户查看页面层面的访问数据统计与趋势，分析 PV、UV 与点击率等指标，评估页面化展示的使用情况。
触发条件	需要评估某一空间/应用下页面化展示入口的访问与转化效果。
前置条件	1. 页面曝光与点击等事件已采集。2. 指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入数据洞察模块并选择时间范围与空间/应用。2. 系统展示页面 PV/UV/点击率趋势与 TopN。3. 用户按页面/组件维度筛选并下钻查看访问或点击明细。
替代流程	1. 若数据量过大，系统按时间粒度或分页方式展示明细。

表 3-22: 会话维度分析用例描述表

ID	DI-02
用例名称	会话维度分析
参与者	平台负责人，业务负责人
用例描述	用户在会话维度对相似 Query 进行聚类分析，查看会话数量、模型识别解决率、意向转人工率与转人工率等指标，定位高频问题与异常会话簇。
触发条件	需要从会话视角评估智能问答链路效果或定位问题分布。
前置条件	1. 会话数据与问答链路事件已采集。2. 会话聚类结果与指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入会话维度分析视图并选择时间范围与空间/应用。2. 系统展示整体会话指标与聚类列表。3. 用户选择某一聚类下钻查看聚类内的 Query 与会话明细。
替代流程	1. 用户可按时间粒度、聚类标题或转人工率等条件筛选聚类。

表 3-23: 内容维度分析用例描述表

ID	DI-03
用例名称	内容维度分析
参与者	业务负责人，业务运营
用例描述	用户从内容视角查看内容的被召回次数、被生成次数、意向转人工次数与转人工次数等指标，评估内容在智能问答场景中的实际使用效果。
触发条件	需要评估内容体系的使用效果并识别高价值或低效内容。
前置条件	1. 召回与生成链路事件已采集并关联到内容。2. 内容维度指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入内容维度分析视图并选择时间范围与空间/应用。2. 系统展示内容指标列表与 TopN。3. 用户选择某一内容下钻查看命中的会话或 Query 明细。
替代流程	1. 用户可按内容类型、类目与标签筛选内容集合。

表 3-24: 会话明细用例描述表

ID	DI-04
用例名称	会话明细
参与者	平台负责人，业务负责人，业务运营
用例描述	用户查看单个会话的详细信息，包括聚类类型、原始 Query、改写 Query、召回内容以及是否转人工等信息，用于定位会话异常与内容支撑问题。
触发条件	需要对某一聚类或某一异常指标进行下钻分析与问题定位。
前置条件	1. 会话事件已采集并能关联到会话 ID。2. 会话明细数据可按权限查询。
后置条件	无
基本流程	1. 用户从会话维度分析或内容维度分析页面下钻进入会话明细。2. 系统展示会话的原始 Query 与改写 Query、召回内容、回复内容与转人工信息。3. 用户按时间轴查看多轮交互细节。
替代流程	1. 若用户无权限查看敏感字段，系统对用户标识与原始文本等字段脱敏展示。

3.2.6 系统非功能需求

在广告与商业化场景下，内容平台面临内容规模大、更新频繁、应用链路长且跨系统依赖多等特点，除功能需求外还需满足一系列非功能需求。

（1）一致性与可追溯性。内容在发布前后存在草稿、待审批、已发布等状态，平台需保证状态流转的正确性；在问答与洞察场景中，通过会话 ID 与链路标识将问题输入、改写、召回证据、生成结果与转人工等事件关联，支撑会话明细下钻与问题定位。

（2）安全与权限。平台在空间与应用维度进行权限隔离，并在内容粒度通过 content_auth 实现策略式授权；平台管理权限采用“角色-权限-作用域”的 RBAC 模型。

（3）可扩展性。平台以统一内容模型承接多形态内容（knowledge、faq、course、case），并通过类目、标签与应用引用机制支撑内容组织与复用；治理规则与洞察指标按模块化方式演进，降低新增形态与新增治理策略的改造成本。

（4）性能与稳定性。平台需同时支撑运营侧高频编辑发布与用户侧高并发浏览检索。在读链路中需以“发布口径”为准并尽量复用缓存与索引能力，以降低数据库压力；在写链路中通过事件机制将索引更新、统计计算等耗时操作异步化处理，避免阻塞主链路；同时建设必要的监控告警与降级策略，保证在异常场景下仍可提供可用的内容查询与问答服务。

3.3 系统整体设计

平台整体架构如图**??**所示。平台围绕内容全生命周期，将核心业务能力划分为内容管理、内容应用、内容治理与数据洞察四个模块，并通过统一内容模型与权限体系实现协同：内容管理负责空间与应用组织、内容创建维护、content_auth 权限控制以及审批发布；内容应用面向页面化场景提供展示与检索，并面向上游智能助手提供检索增强生成（RAG）能力；内容治理对重复度、歧义度与覆盖度等质量问题进行检测，并通过线上化治理任务形成闭环；数据洞察通过事件采集与指标查询评估内容使用效果并反哺治理策略。跨模块的关键协同采用事件机制：内容发布后由内容服务发布更新事件到 MQ，供下游索引更新与统计链路消费，实现主链路与异步处理解耦。

为便于从不同角度理解平台的整体设计，本文在系统架构图基础上，引入逻辑视图、开发视图与物理视图对平台进行补充说明。逻辑视图强调业务能力与闭环关系，开发视图强调工程模块划分与依赖关系，物理视图强调部署形态与基础设施选型。

（1）逻辑视图。图**??**为平台的逻辑视图，用于从业务能力与数据能力角度描述系统的主要功能模块及其协作关系。平台在逻辑上由内容管理、内容应用、内容治理与数据洞察四个核心域构成：内容管理负责空间与应用组织、内容生产维护、内容权限（content_auth）与审批发布，将 knowledge、faq、course 与 case 等多形态内容纳入统一内容资产；内容应用面向门户与管理后台提供展示与检索能力，并面向上游智能助手提供生成式问答能力，支持返回可追溯的引用来源；内容治理对内容质量进行重复、歧义与覆盖等维度的检测，并通过治理任务推动修订与复检；数据洞察对访问、检索与问答行为进行统计分析，评估内容效果与治理收益并反哺运营策略。四个核心域以统一内容模型作为数据输入口径，并通过发布事件与任务事件实现跨域协同，形成“生产—发布—应用—治理—评估”的闭环体系。

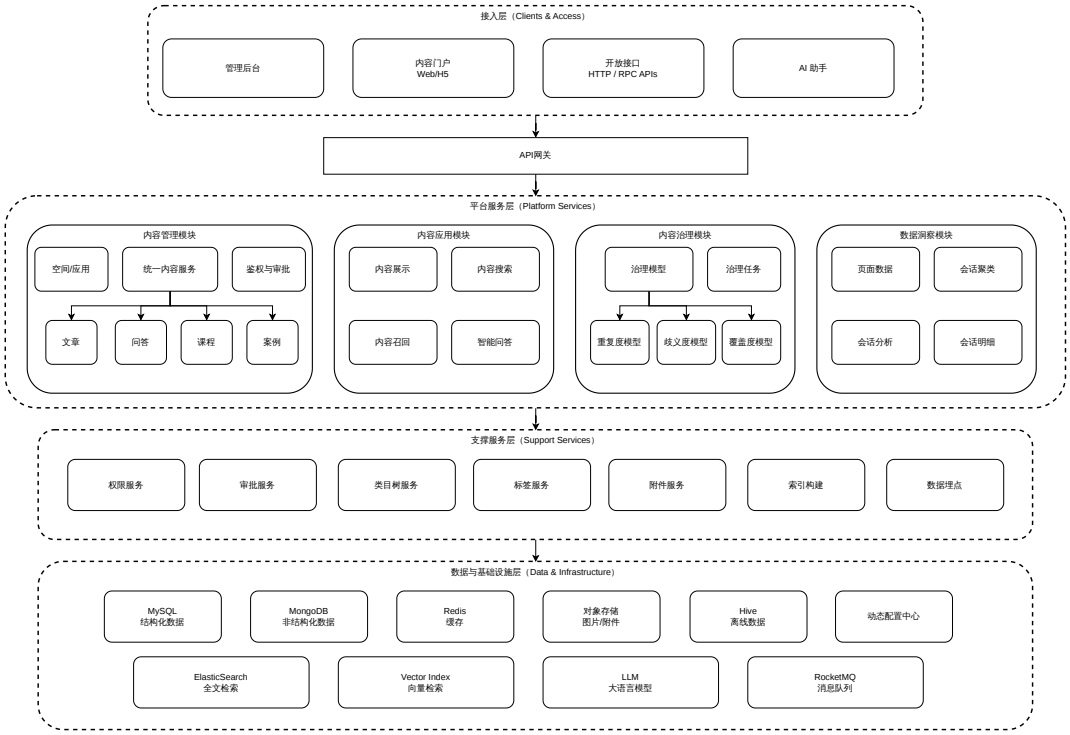

图 3–5: 系统架构图

系统逻辑视图占位（待补充）

图 3–6: 系统逻辑视图

（2）开发视图。图**??**为平台的开发视图，用于描述系统在工程实现层面的模块划分与依赖关系。平台对外提供管理后台、内容门户等入口；统一鉴权由权限拦截器与 RBAC 模块承接，鉴权以 `user_id + scope + code` 为输入进行缓存校验。核心业务能力按域拆分为内容管理、内容应用、内容治理与数据洞察等模块，并复用审批流转、类目与标签、content_auth 等通用能力。为降低耦合与提升可维护性，平台将发布后的索引更新等耗时操作以事件方式异步化处理：内容服务在内容状态变为已发布后发布更新事件到 MQ，下游消费事件并更新 ES 与 Embedding 索引；内容治理在检测与任务处理后推动内容修订并进入再次审批发布；数据洞察侧承接埋点事件并对接数仓产出指标。

开发视图占位（待补充）

图 3-7: 开发视图

（3）物理视图。图**??**为平台的物理视图，用于描述系统在部署与运行环境中的组件构成。平台的管理后台与内容门户以 Web 应用形式部署，后端按模块提供内容管理、内容应用、内容治理与数据洞察等服务能力。数据存储方面，平台使用关系型数据库存储空间、应用、内容元数据、内容权限（content_auth）、审批与治理任务等结构化数据；案例（case）详情存储在 MongoDB 中以适配灵活字段。检索侧同时具备 ES 全文检索索引与 Embedding 语义索引，以支撑页面检索与语义召回。内容发布后由内容服务发布更新事件到 MQ，支撑索引更新等下游处理解耦。数据洞察方面，平台在在线链路采集页面与问答事件，并对接离线数仓产出聚合指标供展示分析。

物理视图占位（待补充）

图 3-8: 物理视图

3.4 系统的模块设计

3.4.1 内容管理模块设计

内容管理模块是平台的内容资产底座，对应需求分析中的“内容管理功能需求”部分，负责空间与应用组织、内容生产与维护、内容权限（content_auth）控制以及审批发布等能力。该模块的设计目标是以统一数据模型纳管多形态内容，并通过可追溯的权限与流程机制，保障内容从生产到发布的过程可控、结

果可查，为后续内容应用与内容治理提供一致的数据输入。以下对内容管理模块的主要功能子模块进行详细设计。

（1）空间与应用管理。空间与应用是内容管理模块的组织基础。平台采用“空间—应用”二级组织模型：空间作为业务归属与权限边界，应用作为内容消费场景与配置边界。空间管理主要包含空间创建、成员管理与默认策略配置；应用管理主要包含应用创建、能力开关与内容范围配置等。空间与应用管理的关键交互过程如图**??**所示。

空间与应用管理时序图占位（待补充）

图 3-9: 空间与应用管理时序图

（2）统一内容模型与多形态内容。平台在内容层引入统一内容模型，使用 content_type 区分 knowledge、faq、course 与 case 四类内容形态，并由一张内容表（content）维护所有内容的公共元数据字段（内容 ID、类型、标题、摘要、正文引用、创建时间、更新时间、状态等）。不同内容类型的具体属性存储在对应的子表中，如 knowledge、faq、course 等；其中 case 为适配结构化字段灵活演进，其详情存储在 MongoDB 中。该设计保证了多形态内容在管理侧具备一致的生命周期操作，同时又允许不同内容形态按需扩展字段。平台核心领域对象及其关联关系可用类图表达，如图**??**所示。

平台核心领域模型类图占位（待补充）

图 3-10: 平台核心领域模型类图

（3）内容权限与审批发布。平台在空间与应用维度进行访问隔离，并在内容粒度通过 content_auth 实现策略式授权，以满足跨应用引用与精细化操作

内容创建与维护时序图占位（待补充）

图 3-11: 内容创建与维护时序图

内容状态机图占位（待补充）

图 3-12: 内容状态机图

控制需求。对于管理端操作，平台结合 RBAC 角色体系完成基础权限控制；对于内容对象，平台通过“授权对象 + 策略 + 变量 + 权限码”的模型表达细粒度权限。RBAC 权限矩阵如表 **??**所示。鉴权决策流程与审批发布链路分别如图 **??**与图 **??**所示。

表 3-25: RBAC 权限矩阵表

角色（作用域）	全局配置/空间创建	空间内应用/成员/审批/发布	内容增删改/提审
平台负责人（GLOBAL）	✓	✓	✓
业务负责人（SPACE）		✓	✓
业务运营（APP|SPACE）			✓

权限体系在工程实现中由 RBAC 与 content_auth 两部分叠加组成：RBAC 负责模块与操作级的基础权限，content_auth 负责内容对象级的策略授权。两类对象及其关系可用类图表达，如图 **??**所示。

在内容发布环节，平台遵循“编辑—提审—审批—发布”的流转机制：内容上线前需要通过审批流程，审批通过后内容服务更新内容状态为已发布，并发布内容更新事件到消息队列，供下游索引更新与相关统计链路异步消费，以降低主链路耦合并提升整体吞吐。

审批与发布链路涉及内容状态、审批记录与发布事件等核心对象，其数据

权限与授权领域类图占位（待补充）

图 3-13: 权限与授权领域类图

内容权限鉴权流程图占位（待补充）

图 3-14: 内容权限鉴权流程图

关系与状态约束可用类图表达，如图**??**所示。

（4）内容组织与应用引用。为提升内容可发现性与复用效果，平台提供类目管理、标签管理与应用引用内容机制。类目以树状层级构建稳定导航入口，标签以扁平维度实现横向聚合；应用引用内容用于表达应用可使用哪些已

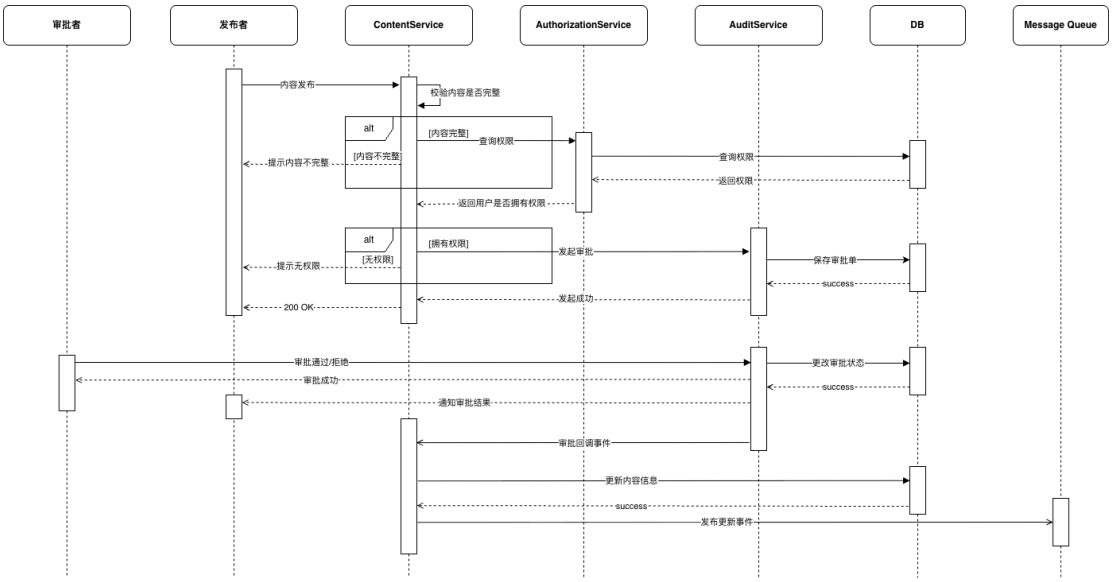

图 3-15: 内容发布流程时序图

审批与发布领域类图占位（待补充）

图 3-16: 审批与发布领域类图

发布内容，使权威口径能够在多个应用复用并保持一致。应用引用内容的关键交互过程如图 **??**所示。

应用引用内容时序图占位（待补充）

图 3-17: 应用引用内容时序图

3.4.2 内容应用模块设计

内容应用模块面向内容消费场景提供统一的展示、检索与问答能力，对应需求分析中的“内容应用功能需求”部分。该模块以内容管理模块产出的上线状态内容作为输入，一方面为内容门户与管理后台提供页面化的内容展示与搜索能力；另一方面为上游智能助手提供生成式问答接口，实现“检索增强生成（RAG）+ 引用可追溯”的智能问答能力。以下对内容应用模块的主要功能子模块进行详细设计。

（1）内容展示。内容展示能力主要服务于内容门户与管理后台等页面化场景。平台基于空间与应用提供内容目录导航、列表聚合与详情页展示能力。目录导航以类目树为主干，配合标签体系提供横向聚合入口；列表聚合支持按时间、热度与业务规则进行排序；详情页展示需呈现内容来源与关联内容，保证用户能够理解当前口径并快速跳转至相关内容。内容展示的主要交互过程如图 **??**所示。

内容展示流程/页面效果图占位（待补充）

图 3-18: 内容展示流程示意图

（2）内容检索。内容检索能力主要服务于关键词检索与结构化筛选。平台支持对标题、摘要、正文与结构化字段进行组合检索，并保证检索结果严格受空间、应用与 content_auth 的权限控制。检索链路由请求解析、范围确定、权限过滤、召回与排序、结果组装与高亮等步骤构成；其中范围确定负责将“空间/应用配置”与“调用方身份”映射为可搜索范围，权限过滤负责对召回候选进行二次过滤以避免越权结果暴露。内容检索的链路示意如图**??**所示。

内容搜索链路示意图占位（待补充）

图 3-19: 内容搜索链路示意图

（3）索引构建。索引构建是连接内容管理与内容应用的关键环节，其目标是在内容发布后将可用内容同步为面向检索与问答的索引结构。平台在发布事件驱动下触发索引构建流程，并针对不同内容形态生成可检索的语义单元（切片）并携带必要元数据。对于页面检索场景，平台构建 ES 全文检索索引以支持关键词匹配；对于生成式问答场景，平台基于切片构建 Embedding 向量索引以支持语义召回。索引构建流程如图**??**所示。

为保证索引与内容状态的一致性，索引构建过程需要以已发布内容为输入，并在构建完成后对外刷新可用索引，避免出现检索结果与内容展示不一致的问题。

索引构建涉及切片（Slice）、全文索引文档、向量索引向量与索引元信息等关键对象，并需要携带空间、应用与权限元数据以支撑范围过滤与权限校验，其对象关系可用类图表达，如图**??**所示。

切片与索引领域类图占位（待补充）

图 3-20: 切片与索引领域类图

索引构建流程示意图占位（待补充）

图 3-21: 索引构建流程示意图

（4）生成式问答。为支撑智能问答场景，平台提供面向生成模型的可控内容支撑能力，其关键在于将内容转化为可检索的语义单元（切片）并在结果中返回可定位的来源信息。平台在索引构建阶段对不同内容形态执行切片策略：文章按固定长度切分并尽量保持段落语义完整；问答以问题与答案作为完整语义单元分别入库并建索引；课程可将每个课节作为一个切片；案例以每个案例作为一个切片。对于图片类内容，使用多模态 Embedding 模型并结合大模型生成文本化理解结果入库；对于表格类内容，避免纯文本切分并按行保持语义完整，同时在切片中携带表头信息。切片索引时携带所属内容 ID、应用 ID 与内容权限等元数据，用于召回阶段的权限过滤与范围控制。索引侧同时构建全文检索索引与语义向量索引，两条通道互补以兼顾精准匹配与语义召回。

在问答阶段，平台采用多通道召回与重排流程：首先对用户问题进行改写以提升召回质量；随后分别在全文检索（ES）与语义向量索引通道召回候选切片，并结合 TopK 与最小相似度阈值（MinScore）进行过滤；最后对候选切片去重、重排并输出证据集合供大模型生成回答。为便于后续治理与评估，问答链路记录召回通道、得分、排序依据与引用来源，并在返回结果中携带内容标识与片段定位信息。生成式问答的召回与重排过程如图 **??**所示。

生成式问答链路涉及问题输入、证据集合与引用结果等核心对象，例如

AskReq、EvidencePack、SliceHit 与 Citation 等，其对象关系与关键字段约束可用类图表达，如图 **??**所示。

生成式问答领域类图占位（待补充）

图 3-22: 生成式问答领域类图

RAG 召回与重排流程示意图占位（待补充）

图 3-23: 内容召回与重排流程示意图

3.4.3 内容治理模块设计

内容治理模块面向内容质量问题构建“发现问题—任务化闭环—复检回收”的治理体系，围绕重复度、歧义度与覆盖度三个方向，将质量问题从结果侧可观测化，并转化为可定位、可执行、可统计的治理工作，最终反哺内容资产与检索问答链路，形成持续演进机制。

（1）治理目标与度量。内容质量体系从内容覆盖、内容质量与效果评估三个方向建立指标口径，并结合客观质量与主观反馈实现质量度量。在内容覆盖方面，关注在给定阈值下用户原声是否能够召回到相关知识；在内容质量方面，关注内容重复与口径冲突带来的冗余与歧义；在效果评估方面，关注内容在检索与问答链路中的命中、引用与会话解决效果等指标，为治理优先级与策略调整提供依据。

（2）重复度/歧义度模型。重复度与歧义度检测的核心思路是采用“候选召回 + 大模型判别”的方式避免全量两两比较。首先对每个待检测内容在检索侧

召回 TopK 相似候选，形成候选对集合；随后对“当前内容—候选内容”构造判别提示词，使用大模型输出相似度与歧义度判断；最后将判别结果汇总为相似对、重复组与歧义候选组，并输出来源定位信息供人工复核与任务生成。重复度与歧义度检测流程如图 **??**所示。

重复度/歧义度模型检测流程示意图占位（待补充）

图 3–24: 重复度/歧义度模型检测流程示意图

（3）覆盖度治理机制。覆盖度治理用于持续发现“缺知识”与薄弱领域，并将缺口转化为可闭环的补充任务。其一，基于用户原声的覆盖治理以真实会话为驱动：在设定会话周期与召回阈值的前提下，对用户原声执行召回评估，召回结果未满足阈值要求的视为缺知识；对缺知识原声进行聚类，将相似原声聚为一类生成覆盖任务，由业务人员补充内容或完善现有内容，并通过每日定时巡检对任务范围进行复核，避免“内容已补充但任务未更新”导致统计失真。其二，基于内容先验的覆盖治理接入广告与商业化知识体系知识图谱，对比知识体系中的类目/标签/概念/定义与现有内容覆盖情况，发现缺失或薄弱领域并进入任务闭环。

（4）质量治理任务线上化。为将质量治理从离线跑数与人工推动升级为线上闭环，平台将治理过程抽象为主任务与子任务两层结构。主任务负责一次治理计算的配置与状态管理，包含任务名称、空间、会话周期与召回阈值等上下文信息；主任务进入计算中后生成治理任务列表，并在计算完成后对外展示结果。子任务主要包含覆盖任务与重复歧义任务两类：覆盖任务承载缺知识聚类结果，支持人工标记“无需补充/需要补充”并填写说明，系统每日巡检若发现已可召回到相关答案则自动完成；重复歧义任务承载重复对与歧义对结果，支持人工判断是否需要下线并完成处理标记，系统每日巡检若发现内容变更、下线或删除后不再存在重复/歧义，则自动完成。质量治理任务线上化的结构与关键流程如图 **??**所示。

治理任务线上化涉及主任务、覆盖任务、重复/歧义任务以及任务处理记录等对象，并需表达任务状态机与关联关系，其领域模型类图如图 **??**所示。

质量治理任务线上化流程示意图占位（待补充）

图 3-25: 质量治理任务线上化流程示意图

治理任务领域类图占位（待补充）

图 3-26: 治理任务领域类图

（5）复检回收与链路一致性。治理任务处理通常会引发内容修订与再次发布，平台需要保证“内容资产—索引—问答结果”在治理后保持一致。对于需要修订的内容，治理处理应与内容管理的提审与发布流程打通，发布后由索引构建服务消费发布事件完成增量索引更新；随后对任务范围执行复检回收，确认质量问题是否消除，并将复检结果与指标变化回收至数据洞察体系，用于形成治理前后对比与策略迭代。跨域协同链路示意如图**??**所示。

治理修订-发布-复检时序图占位（待补充）

图 3-27: 治理修订-发布-复检时序图

3.4.4 数据洞察模块设计

数据洞察模块为平台提供效果评估与问题定位能力，是“应用驱动生产、数据反哺治理”的关键闭环环节。该模块的实现链路采用“前端埋点与服务端事件

采集 → 数仓统一加工与指标口径沉淀 → 洞察服务查询与权限控制 → 前端可视化与下钻明细”的模式：数据加工与指标口径由数仓侧负责，本平台侧重点在事件规范、采集链路、与数仓/BI 的对接以及查询服务建设。数据采集与指标生成链路示意如图 **??**所示。

数据采集与指标生成链路示意图占位（待补充）

图 3–28: 数据采集与指标生成链路示意图

（1）数据采集。数据采集需要覆盖页面化消费链路与智能问答链路两类场景。在页面化链路中，平台采集曝光、点击、搜索输入与结果点击等行为数据；在问答链路中，平台采集问题输入、召回证据、生成结果与转人工等链路数据。为避免采集口径不一致导致分析偏差，平台对埋点事件与字段进行统一定义，例如 page_view、click、query_submit、rag_recall、llm_generate、handoff_intent 与 handoff_success 等，并统一携带 app_id、space_id、session_id、trace_id 与 content_id 等公共维度字段，便于端到端追踪与明细还原。

为支撑事件的可扩展与跨链路关联，平台将事件抽象为“公共维度 + ext 扩展字段”的统一模型，并以 trace_id 与 session_id 建立会话与链路关联。事件数据模型与关键维度关系如图 **??**所示。

事件采集与维度模型类图占位（待补充）

图 3–29: 事件采集与维度模型类图

（2）指标对接与查询服务。在指标体系设计上，平台从页面、会话与内容三个视角承接数仓侧产出的聚合指标：页面数据监控关注 PV/UV/点击率；会话维度分析关注会话数量、模型识别解决率、意向转人工率与转人工率；内

容维度分析关注内容被召回次数、被生成次数以及转人工相关指标。平台在洞察服务中提供按时间范围、空间/应用、渠道与页面等维度的只读查询接口，支持从聚合图表下钻到会话明细，并通过 RBAC 与字段脱敏控制明细可见范围。

（3）会话维度分析与聚类。会话维度分析以用户会话为单位组织数据，并对相似 Query 进行聚类，以支持“主题簇（Cluster）”口径下的统计与下钻。聚类构建逻辑采用基于语义相似图的聚类方法：

1. 对历史 Query 进行向量化编码，写入向量检索索引。
2. 设定 `min_score` 阈值，相似度 ≥ 阈值的 Query 之间建立语义连接。
3. 构建相似图：每个 Query 为节点，相似度满足阈值则建立边。
4. 从未归类 Query 出发，递归召回满足阈值的 Query 直到无新增节点，形成一个聚类。
5. 使用大模型总结主题，可清洗低相关 Query 并生成聚类标题（主题标签）。
6. 重复上述过程，直到所有 Query 被聚类。
7. 对新增 Query 判断是否归入已有聚类，若匹配则归入，否则创建新聚类。

会话维度指标体系包括聚类级与整体级两部分：聚类级指标包含聚类标题、聚类会话数、聚类 Query 数、模型识别解决率、意向转人工率与实际转人工率；整体级指标包含整体会话数、整体 Query 数、整体模型识别解决率、整体意向转人工率与整体实际转人工率。上述指标均支持按天/周/月/季或自定义时间范围进行筛选与对比。

（4）内容维度分析。内容维度分析用于评估内容在 RAG 链路中的实际使用效果与质量表现，统计内容被召回、被生成以及转人工相关指标。内容级指标包含内容 ID、内容标题、内容类型、所属类目、被召回会话数、被生成会话数、意向转人工会话数、转人工会话数与时间范围；整体指标包含整体内容数、有召回内容数与有生成内容数，并支持按时间范围与内容类型筛选。

（5）会话明细。会话明细用于追溯单个会话的上下文与决策链路，支持查看聚类类型、原始 Query、改写 Query、召回内容以及是否转人工等信息。会话明细字段包含会话 ID、聚类类型、用户原声、改写内容、召回内容（ID 与标题）、回复内容、意向转人工、是否转人工、用户 ID、用户类型与会话时间，并受 RBAC 与字段脱敏策略控制。

3.5 数据库设计

平台的持久化存储围绕“统一内容资产 + 权限控制”展开：空间、应用、内容元数据、审批流转与内容权限（content_auth）等结构化数据落在关系型数据库中；案例（case）的详情数据因字段结构灵活存储在 MongoDB 中。检索与问答链路使用全文检索与语义向量两类索引以支撑页面检索与 RAG 召回；数据洞察的离线聚合结果由数仓侧产出，平台侧以查询接口承接并展示。

3.5.1 概念结构设计

平台以空间与应用作为组织边界：空间（Space）用于区分不同业务线并形成权限边界，应用（Application）用于表示空间下的具体业务领域或使用场景。内容（Content）作为统一抽象实体，通过内容类型字段区分 knowledge、faq、course 与 case 四类内容形态，并与类目、标签、应用引用关系共同构成内容组织方式。权限体系由两部分叠加：一是平台管理权限，采用“角色-权限-作用域”的 RBAC 模型；二是内容级可见性与可操作范围，由 content_auth 模块在内容粒度再判一层，以支持免登录可见、员工/部门/角色匹配等策略。

数据库 E-R 图占位（待补充）

图 3–30: 数据库 E-R 图

3.5.2 表结构设计

本节给出平台数据库关键表结构。表结构与字段口径以项目手册为准：内容资产以空间与应用为组织边界，内容采用统一内容模型承接 knowledge、faq、course、case 四类形态；权限采用 RBAC 与 content_auth 两层叠加；内容上线依赖审批表记录审批状态；跨应用复用通过 content_app_ref 维护引用关系。

（1）空间与应用表。空间与应用是内容组织与权限隔离的基础边界，其表结构如表 **??**与表 **??**所示。

表 3-26: 空间表（space）

字段名	字段中文名	字段类型	说明
id	空间 ID	bigint	主键
name	空间名称	varchar(255)	业务标识
owner_id	负责人 ID	bigint	平台用户标识
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-27: 应用表（application）

字段名	字段中文名	字段类型	说明
id	应用 ID	bigint	主键
space_id	空间 ID	bigint	外键，关联 space
name	应用名称	varchar(255)	业务标识
type	应用类型	varchar(50)	门户/智能助手等
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

（2）统一内容表。内容表承接所有内容形态的公共字段，其表结构如表 **??**所示。

（3）多形态内容子表。不同内容类型的差异字段存储在对应子表中：knowledge/faq/course 落在关系型数据库中，case 详情存储在 MongoDB 中。其中 faq 除标准问与答案外，还需要维护相似问集合以支撑相似问召回与口径统一。相关表结构如表 **??**至表 **??**所示。

（4）类目与标签表。为支撑内容在页面化展示与检索场景下的组织与聚合，平台提供类目树与标签体系，并通过关联表建立内容与类目/标签的绑定关系。相关表结构如表 **??**至表 **??**所示。

（5）平台 RBAC 表。平台管理权限使用五张表落地：user、role、permission、role_permission、user_role_binding。用户-角色绑定关系区分 GLOBAL/SPACE/APP 作用域，以实现“平台负责人（GLOBAL）、业务负责人（SPACE）、业务运营（APP|SPACE）”的权限隔离与授权。其表结构如表 **??**至表 **??**所示。

表 3-28: 内容表（content）

字段名	字段中文名	字段类型	说明
id	内容 ID	bigint	主键
space_id	空间 ID	bigint	外键，关联 space
app_id	归属应用 ID	bigint	内容创建所在应用
content_type	内容类型	varchar(50)	knowledge/faq/course/case
title	标题	varchar(255)	
summary	摘要	varchar(1024)	
status	状态	varchar(50)	DRAFT/PENDING_REVIEW/PUBLISHED/OFFLINE
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-29: 知识内容表（knowledge）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
body	正文	longtext	富文本/Markdown 等

表 3-30: 问答内容表（faq）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
question	标准问题	varchar(1024)	
answer	答案	longtext	

表 3-31: 相似问表（faq_similar_question）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
faq_id	问答 ID	bigint	外键，关联 faq
question	相似问	varchar(1024)	
create_time	创建时间	datetime	

表 3-32: 课程内容表（course）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
cover_url	封面	varchar(1024)	可选
level	难度等级	varchar(50)	可选

表 3-33: 课节表（course_lesson）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
course_id	课程 ID	bigint	外键，关联 course
title	课节标题	varchar(255)	
lesson_no	课节序号	int	课程内排序
content	课节内容	longtext	文本/字幕等

表 3-34: 案例详情集合（MongoDB：case_detail）

字段名	字段中文名	字段类型	说明
_id	主键 ID	ObjectId	MongoDB 主键
content_id	内容 ID	bigint	关联 content.id
detail	详情字段	json	其余结构化/非结构化字段

表 3-35: 标签表（tag）

字段名	字段中文名	字段类型	说明
id	标签 ID	bigint	主键
app_id	应用 ID	bigint	标签所属应用
name	标签名称	varchar(255)	
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-36: 内容-标签关联表（content_tag_ref）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
tag_id	标签 ID	bigint	外键，关联 tag
create_time	创建时间	datetime	

表 3-37: 类目表（category）

字段名	字段中文名	字段类型	说明
id	类目 ID	bigint	主键
app_id	应用 ID	bigint	类目所属应用
parent_id	父类目 ID	bigint	根节点为 0 或空
name	类目名称	varchar(255)	
path	路径	varchar(1024)	祖先链路编码，可选
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-38: 内容-类目关联表（content_category_ref）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
category_id	类目 ID	bigint	外键，关联 category
create_time	创建时间	datetime	

表 3-39: 用户表（user）

字段名	字段中文名	字段类型	说明
id	用户 ID	bigint	主键
name	用户名	varchar(255)	
status	状态	varchar(50)	ENABLED/DISABLED

表 3-40: 角色表（role）

字段名	字段中文名	字段类型	说明
id	角色 ID	bigint	主键
name	角色名称	varchar(255)	平台负责人/业务负责人/业务运营等

表 3-41: 权限表（permission）

字段名	字段中文名	字段类型	说明
id	权限 ID	bigint	主键
code	权限码	varchar(255)	如 SPACE_CREATE、CONTENT_PUBLISH
name	权限名称	varchar(255)	可读描述

（6）内容权限表（content_auth）。内容级权限以“内容 + 策略 + 变量 + 权限码”的方式表达，其表结构如表 **??**所示；auth_tactic 支持免登录可见、所有外部用户可见、所有内部员工可见、员工匹配、部门匹配、角色匹配以及自定义匹配等策略。

（7）内容审批与应用引用表。内容上线依赖审批表记录审批状态与审批意见；跨应用复用通过 content_app_ref 维护引用关系。其表结构如表 **??**与表 **??**所示。

（8）治理任务表。内容治理模块将检测结果以治理任务形式线上化流转，主任务负责配置与状态管理，重复歧义任务与覆盖度任务承载具体治理问题。其表结构如表 **??**至表 **??**所示。

（9）数据洞察数仓表。数据洞察模块的指标计算与口径沉淀由数仓侧负责，平台侧通过查询接口对接数仓侧宽表与明细表。为保证论文写作与实现口径一致，本节给出数据洞察侧核心数据表（数仓侧）的字段设计，如表 **??**至

表 3-42: 角色-权限关联表（role_permission）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
role_id	角色 ID	bigint	外键，关联 role
permission_id	权限 ID	bigint	外键，关联 permission

表 3-43: 用户-角色绑定表（user_role_binding）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
user_id	用户 ID	bigint	外键，关联 user
role_id	角色 ID	bigint	外键，关联 role
scope_type	作用域类型	varchar(20)	GLOBAL/SPACE/APP
scope_id	作用域 ID	bigint	空间 ID/应用 ID，可为空

表 3-44: 内容权限表（content_auth）

字段名	描述	类型	备注
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	被授权对象的唯一标识
auth_tactic	权限策略	varchar(50)	策略类型（如员工匹配、部门匹配等）
auth_variable	权限变量	varchar(200)	策略变量值（如员工 ID、部门 ID 等）
auth_code	操作权限	varchar(20)	权限码集合（如 r/w/u 等）

表 **??**所示。

表 3-45: 内容审批表（content_approval）

字段名	字段中文名	字段类型	说明
id	审批单 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
submitter_id	提交人 ID	bigint	平台用户标识
reviewer_id	审批人 ID	bigint	平台用户标识
status	审批状态	varchar(50)	PENDING/APPROVED/REJECTED
comment	审批意见	varchar(1024)	可选
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-46: 应用引用表（content_app_ref）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
source_app_id	引用源应用 ID	bigint	外键，关联 application
target_app_id	引用目标应用 ID	bigint	外键，关联 application
content_id	内容 ID	bigint	外键，关联 content
create_time	创建时间	datetime	

表 3-47: 主任务表（govern_main_task）

字段名	字段中文名	字段类型	说明
id	主任务 ID	bigint	主键
name	任务名称	varchar(255)	
space_id	业务空间	bigint	外键，关联 space
session_period	会话周期	varchar(50)	天/周/月等
recall_threshold	召回阈值	decimal(10,4)	相似度/覆盖阈值
status	状态	varchar(50)	计算中/计算完成/已完成等
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-48: 重复歧义任务表（govern_dup_amb_task）

字段名	字段中文名	字段类型	说明
id	任务 ID	bigint	主键
main_task_id	主任务 ID	bigint	外键，关联 govern_main_task
app_id	应用 ID	bigint	外键，关联 application
task_type	子任务类型	varchar(50)	DUPLICATE/AMBIGUOUS
content_id_a	内容 A	bigint	外键，关联 content
content_id_b	内容 B	bigint	外键，关联 content
status	状态	varchar(50)	TODO/DONE 等
assignee_id	责任人	bigint	平台用户标识
result	处理结果	varchar(255)	下线/保留/合并等
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-49: 覆盖度任务表（govern_coverage_task）

字段名	字段中文名	字段类型	说明
id	任务 ID	bigint	主键
main_task_id	主任务 ID	bigint	外键，关联 govern_main_task
app_id	应用 ID	bigint	外键，关联 application
item_type	覆盖对象类型	varchar(50)	类目/标签/概念/定义/query 等
item_value	覆盖对象值	varchar(1024)	名称或标识
score	覆盖得分	decimal(10,4)	可选
status	状态	varchar(50)	TODO/DONE 等
assignee_id	责任人	bigint	平台用户标识
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-50: 事件明细表（ODS：di_event_ods）

字段名	字段中文名	字段类型	说明
event_name	事件名	string	page_view/click/query_submit/rag_recall 等
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
user_id	用户 ID	bigint	可为空
visitor_id	匿名访客 ID	string	可为空
session_id	会话 ID	string	
trace_id	链路标识	string	
content_id	内容 ID	bigint	可为空
timestamp	时间戳	datetime	
channel	渠道	string	可为空
device	设备	string	可为空
ext	扩展字段	json	事件特有字段

表 3-51: 页面指标宽表（DWS：di_page_metric_d）

字段名	字段中文名	字段类型	说明
dt	统计日期	date	分区字段
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
page_id	页面 ID	string	
pv	PV	bigint	
uv	UV	bigint	
click_cnt	点击数	bigint	
ctr	点击率	decimal(10,4)	可选

表 3-52: 会话聚类指标宽表（DWS：di_session_cluster_d）

字段名	字段中文名	字段类型	说明
dt	统计日期	date	分区字段
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
cluster_id	聚类 ID	string	
cluster_title	聚类标题	string	
session_cnt	会话数	bigint	
query_cnt	Query 数	bigint	
solve_rate	解决率	decimal(10,4)	模型识别解决率
handoff_intent_rate	意向转人工率	decimal(10,4)	
handoff_rate	实际转人工率	decimal(10,4)	

3.6 本章小结

本章围绕面向广告与商业化内容的管理、应用与治理平台展开分析与设计。首先从业务与系统视角给出了平台的整体定位与关键链路，明确平台由内容管理、内容应用、内容治理与数据洞察四个核心域构成，并以统一内容模型、权限体系与事件机制实现协同。随后在需求分析中按照模块给出了核心用例与用例描述表，明确平台在页面展示、内容搜索与智能问答场景下的能力边界，以及治理闭环与效果评估的关键需求。在总体设计与模块设计部分，本章

表 3-53: 内容指标宽表（DWS：di_content_metric_d）

字段名	字段中文名	字段类型	说明
dt	统计日期	date	分区字段
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
content_id	内容 ID	bigint	
content_type	内容类型	string	knowledge/faq/course/case
category_id	类目 ID	bigint	可为空
recall_session_cnt	被召回会话数	bigint	
generate_session_cnt	被生成会话数	bigint	
handoff_intent_session_cnt	意向转人工会话数	bigint	
handoff_session_cnt	转人工会话数	bigint	

表 3-54: 会话明细表（DWD：di_session_detail）

字段名	字段中文名	字段类型	说明
session_id	会话 ID	string	主键之一
cluster_id	聚类 ID	string	可为空
raw_query	用户原声	string	可脱敏
rewritten_query	改写 Query	string	可脱敏
recalled_contents	召回内容	json	含 content_id&title 等
answer	回复内容	string	可脱敏
handoff_intent	意向转人工	tinyint	0/1
handoff_success	是否转人工	tinyint	0/1
user_id	用户 ID	bigint	可脱敏
user_type	用户类型	string	内部/外部/匿名等
session_time	会话时间	datetime	

说明了内容发布后的事件驱动机制与索引构建策略，并给出内容切片、多通道召回与治理任务线上化等关键实现思路。最后在数据库设计部分对平台的权限相关核心数据对象与存储选型进行了说明，为后续系统实现与效果评估奠定基础。

第四章　内容管理、应用与治理平台的实现

4.1　内容管理模块的实现

内容管理模块是平台的内容资产底座，承担内容生产、组织、权限与发布等职责，是后续内容应用与内容治理的数据输入来源。实现层面，模块围绕统一内容模型构建空间与应用组织、内容与版本管理、类目与标签组织、跨应用引用、权限控制与审批发布等能力，并通过事件机制在发布后驱动索引构建与数据回收，保证“发布版本”口径在展示、检索、问答与治理链路中的一致性。

参考师兄论文在实现章节中的组织方式，本节首先给出内容管理模块对外提供的关键接口概览，随后按子模块展开说明其核心实现机制。内容管理模块的关键服务端接口定义如表**??**所示。

表**??**中列出的接口覆盖了内容管理模块的三条主链路：组织边界建立（空间/应用）、内容生产与发布（内容/版本/审批）、以及复用与访问控制（组织/引用/权限）。创建空间与创建应用接口用于初始化业务隔离边界与消费边界，后续所有内容都需归属到空间与源应用中。内容创建、编辑与草稿保存接口用于支撑高频内容生产活动，并通过版本链承载变更历史。提审、审批与发布接口用于将内容从可编辑态切换到对外可用态，并为下游检索问答与治理模块提供“发布版本”的统一输入口径。类目树与标签接口用于组织内容并服务于导航与筛选。应用引用接口用于跨应用复用权威内容并沉淀影响范围。最后，权限相关接口为后续读链路的范围确定与结果过滤提供约束条件，是保证跨应用复用不导致越权暴露的关键。

4.1.1　多形态内容统一建模与管理

多形态内容统一建模是内容管理模块实现的基础。平台将 knowledge、faq、course、case 四类内容抽象为统一的内容实体 content，并在子表中分别维

表 4-1: 内容管理模块接口表

服务名称	接口名称	功能描述
空间/应用管理	createSpace / updateSpace / getSpaceDetail	创建与维护业务空间，作为 界。
空间/应用管理	createApp / updateApp / getAppDetail	在空间内创建应用并配置消 围。
内容管理	createContent / updateContent	创建与编辑内容元数据，写 本链。
内容管理	saveDraft / getDraft	保存草稿并获取可编辑版本 致性控制。
内容管理	submitReview / approve / reject	发起提审、审批通过或驳回 并生成发布口径。
内容管理	publish / offline	发布生成对外可见的发布版 外不可见。
组织能力	upsertCategory / moveCategory / upsertTag	维护应用内类目树与标签体 与聚合展示。
引用能力	addReference / removeReference / getReferenceList	配置应用引用内容关系，支 范围计算。
权限能力	checkPermission / getContentAuth	基于 RBAC 与 content_auth 查询。

护不同形态的结构与正文字段，从而实现“统一元数据口径、差异字段独立演进、发布状态可控”的工程目标。其核心实现要点包括统一元数据字段设计、多形态差异字段承载、面向发布的状态约束以及跨链路可回溯的内容标识体系。

本节围绕“内容创建与编辑、版本链维护、状态流转约束”给出实现说明，并通过关键实现片段支撑论述。内容创建与版本管理的交互过程与状态流转可分别与图 **??**与图 **??**对应理解。

为便于后续内容应用、治理与洞察模块在实现上复用统一口径，内容管理模块在存储层形成了以空间、应用、内容为主干的结构化表设计，并为不同形态提供子表或详情存储。内容管理模块的核心数据表与职责划分如表 **??**所示。

（1）统一内容实体与多形态子表。平台在关系型数据库中维护 content 表与多形态子表：content 表记录空间、归属应用、类型、标题、摘要、状态等元数据；不同内容形态的正文与结构字段落在对应子表中，例如 knowledge.body、faq.question/answer、course_lesson.content。该设计使列表、权限、

表 4-2: 内容管理模块核心数据表

数据对象	关键字段（示例）	说明
space	id, name, owner_id, status	业务归属与隔离边界，承载成员与默认权限策略。
application	id, space_id, name, type, status	内容消费场景与配置边界，定义可用形态、组织方式与范围。
content	id, space_id, app_id, content_type, title, summary, status	统一内容实体元数据，content_type 区分 knowledge/faq/course/case。
knowledge	content_id, body	知识正文（富文本/Markdown），用于页面展示与切片索引。
faq	content_id, question, answer	FAQ 以标准问答对表达最小知识单元。
faq_similar_question	faq_id, question	相似问扩展，用于覆盖不同问法并提升 FAQ 召回。
course / course_lesson	content_id, title, lesson_no, content	课程按“课程元信息 + 课节”拆分，lesson_no 维护顺序。
case_detail(MongoDB)	content_id, detail(json)	案例详情字段灵活演进，采用文档存储承载可变结构。
content_app_ref	source_app_id, target_app_id, content_id	应用引用内容关系，用于跨应用复用与影响范围分析。
content_approval	content_id, submitter_id, reviewer_id, status	内容审批记录，驱动发布状态流转并约束上线口径。
content_auth	content_id, auth_tactic, auth_variable, auth_code	内容粒度策略授权，与 RBAC 共同完成访问控制。

检索与治理能够基于统一的 content 标识协同，同时允许不同形态按需演进字段结构。

（2）内容创建与编辑保存关键实现。内容创建与编辑的核心在于保证跨表写入的原子性，并避免并发编辑导致的数据覆盖。创建时，系统在同一事务内写入 content 元数据，并根据 content_type 写入对应子表的正文或结构字段；编辑保存时，系统采用“读-改-写 + 条件更新”的方式进行一致性控制：客户端携带基于 modify_time 的基线时间戳，写入时在 SQL 层做条件更新校验，若影响行数为 0 则提示刷新后重试。内容创建与草稿保存的关键实现片段如

图 **??**所示。

```
public Long createContent(CreateReq req) {
  Long contentId = contentDao.insertContent(req.spaceId, req.
      appId, req.type, req.title, req.summary, "DRAFT");
  if ("knowledge".equals(req.type)) {
    knowledgeDao.insert(contentId, req.body);
  } else if ("faq".equals(req.type)) {
    faqDao.insert(contentId, req.question, req.answer);
  } else if ("course".equals(req.type)) {
    courseDao.insert(contentId, req.courseMeta);
  } else if ("case".equals(req.type)) {
    caseDao.insertDetail(contentId, req.detailJson);
  }
  return contentId;
}

public Long saveDraft(SaveDraftReq req) {
  boolean ok = contentDao.casUpdateMeta(req.contentId, req.
      baseModifyTime, req.title, req.summary);
  if (!ok) {
    throw new ConflictException("content version changed,
        please refresh");
  }
  if ("knowledge".equals(req.type)) {
    knowledgeDao.updateBody(req.contentId, req.body);
  } else if ("faq".equals(req.type)) {
    faqDao.update(req.contentId, req.question, req.answer);
  }
  return req.contentId;
}
```

图 4-1: 内容创建与版本管理关键代码

页面化展示在实现上采用“元数据轻量返回、正文按需加载”的策略：列表与目录页仅返回 content 表中的 title、summary、content_type 等元数据字段，并携带类目、标签等用于前端渲染筛选项；详情页在完成对象级权限校验后再读取对应形态子表的正文或结构字段，避免大字段在高频列表请求中被重复加载。对于 course 与 case 等结构型内容，详情页由服务端将多条课节记录或 MongoDB detail(json) 聚合为统一的渲染模型，保证前端消费一致。

在性能层面，内容列表查询以 space_id 与 app_id 作为强过滤条件，并结合状态字段仅返回已发布内容；类目与标签维度通过关联表或索引字段进行过滤与排序。对于高频访问的应用目录树与标签集合，服务端可按 (app_id) 维度做短期缓存，以减少重复构建树结构带来的 CPU 开销；当类目或标签发生变更

时，更新接口同步触发缓存失效，保证读链路与配置链路一致。

（3）多形态差异字段落地。不同内容形态的差异不仅体现在“是否有正文”，更体现在内容结构、编辑方式与召回粒度上。knowledge 以长文为主，正文用于页面渲染与段落切片；faq 以问答对为主，并通过 faq_similar_question 维护相似问集合以覆盖不同问法；course 以课节为基本单元，lesson_no 用于稳定排序与学习路径展示；case 字段结构随业务演进变化较快，采用文档存储承载 detail(json) 以降低关系型表结构频繁变更成本。

（4）状态与发布口径约束。内容生命周期至少包含草稿、待审批、已发布、已下线等状态，平台在状态机约束下实现编辑、提审、审批、发布与下线等操作。实现中，读链路对外仅返回已发布状态内容；写链路在提审时冻结变更并进入待审批状态，审批通过后更新内容状态为已发布并触发发布事件，驱动下游索引更新与数据回收，避免“草稿可见”导致的口径漂移。

4.1.2 内容组织与引用机制

内容组织与引用机制用于提升内容的可发现性与复用效率，降低重复维护成本与口径不一致风险。实现层面，平台以“应用内类目树 + 标签体系 + 应用引用内容关系”为核心组织方式：类目树提供稳定的导航入口，标签提供跨类目聚合与筛选能力，引用关系提供跨应用复用的工程化落地。

（1）类目树与标签体系实现。类目树以应用为边界维护，采用父子层级结构表达目录导航关系，并对节点移动、删除等操作进行约束以避免形成环或不合法层级。平台在类目查询场景提供“按应用获取完整树”和“按节点获取路径/子树”等接口，并在内容展示时通过内容-类目关联完成目录聚合。标签体系以扁平集合方式管理，支持标签的新增、禁用与与内容的绑定关系维护，用于在门户与后台提供基于标签的聚合与筛选能力。

在数据结构上，类目表以 parent_id 表达父子关系，并维护 path 字段以表达祖先链路编码，使得“获取节点路径”“批量获取子树”等操作可以在单次查询或少量查询中完成，降低前端渲染目录树的请求开销。对于类目移动，服务端需要校验目标父节点不能位于待移动节点的子树中，并在同一事务内完成 parent_id 与 path 的更新，以避免出现中间态导致目录树不可用。类目移动的关键实现片段如图 **??**所示。

标签体系在实现上通常以 (app_id, name) 作为唯一性约束，避免同一应用下出现同名标签导致筛选歧义。对于历史标签清理，平台采用“禁用”而非“物理

```
public void moveCategory(MoveCategoryReq req) {
  Category node = categoryDao.get(req.categoryId);
  Category newParent = categoryDao.get(req.newParentId);
  if (newParent.path.startsWith(node.path)) {
    throw new BizException("invalid move: cycle detected");
  }
  String newPath = newParent.path + "/" + node.id;
  categoryDao.updateParentAndPath(node.id, req.newParentId,
      newPath);
  categoryDao.batchUpdateSubtreePath(node.path, newPath);
}
```

图 4-2: 类目移动关键代码

删除”的策略，使得历史内容的标签标注能够被稳定回溯，同时在展示与检索场景下仅对启用标签开放筛选入口。

（2）应用引用内容关系实现。应用引用内容用于表达“某应用可使用哪些已发布内容”，其本质是应用与内容的多对多关系。实现中，平台通过 content_app_ref 表维护引用关系：写入引用时校验被引用内容的发布状态与调用方权限，避免将未发布或无权限内容暴露到目标应用；读取内容列表与检索范围确定时，以引用关系作为范围约束之一，从而在应用侧形成可控的内容可见集合。

引用关系在工程上需要满足两个关键约束：其一是写入幂等，避免重复配置导致引用表膨胀或出现重复行；其二是引用边界清晰，引用只影响目标应用可见范围，不改变内容的归属空间与归属应用，从而保证权限、审批与治理责任仍然在内容归属侧闭环。实现中 refDao.upsert 通过唯一键约束或插入忽略的方式保证幂等；在读取侧，范围确定逻辑以 (app_id, space_id) 为强边界，并在启用引用时将引用集合与类目/标签条件组合过滤，确保跨应用复用不会绕过应用侧的组织配置。

为保证“可复用”与“可控”的一致性，引用关系写入并非简单的多对多插入，而需要同时满足内容状态、授权策略与范围配置等约束。引用关系写入的关键实现片段如图 **??**所示。

4.1.3 权限管理与审批流程

内容管理模块同时面向管理后台与对外消费链路提供能力，因此权限体系需要同时覆盖“后台可操作范围”与“内容可见/可操作范围”。平台采用“RBAC +

```
public void addReference(AddRefReq req) {
  authService.checkPermission(req.operator, req.targetAppId,
     "CONTENT_REF_ADD");
  Content c = contentDao.get(req.contentId);
  if (!c.isPublished()) {
    throw new BizException("only published content can be
       referenced");
  }
  refDao.upsert(req.targetAppId, req.contentId);
}
```

图 4-3: 应用引用内容关键代码

content_auth"的组合实现：RBAC 用于在空间与应用维度进行角色授权与模块级/操作级控制，content_auth 用于在内容粒度表达更细的策略约束，以支撑跨应用引用、精细化可见范围以及合规要求。

（1）RBAC 权限控制实现。RBAC 以用户—角色—权限项映射为基础，角色在空间或应用维度生效，权限项覆盖空间/应用配置、内容创建编辑、删除、提审、审批、上下线与查看检索等操作。实现中，接口层在处理请求时首先完成身份校验与 RBAC 权限校验，确保调用方具备对应操作能力，再进入后续对象级策略判断，避免越权调用对系统造成影响。

在鉴权决策上，平台将请求解析为 `user_id + scope_type + scope_id + permission_code` 四元组：scope 用于表达授权生效范围（GLOBAL/SPACE/APP），permission_code 用于表达具体操作。鉴权时先根据请求携带的 space_id、app_id 解析出作用域，再判断用户在该作用域下是否绑定了包含目标权限码的角色，从而实现跨空间隔离与空间内精细授权。

为降低鉴权对高并发读链路的额外开销，平台在接入层对 RBAC 结果进行缓存：以 `user_id + scope_type + scope_id + permission_code` 作为缓存键，缓存值为鉴权是否通过及可选的角色信息。缓存更新遵循"写后失效"的原则：当空间负责人变更成员角色、或平台负责人调整角色-权限关系时，管理侧接口同步触发对应用户的缓存失效，保证权限变更能够在可接受延迟内生效。对于未命中缓存的请求，服务端从 `user_role_binding` 与 `role_permission` 表加载关系并完成裁决，再回填缓存。

（2）content_auth 策略授权实现。针对内容可见性与细粒度动作控制，平台以"内容 + 策略 + 变量 + 权限码"的模型表达策略：策略通过 auth_tactic 与 auth_variable 描述可见范围或动作约束，auth_code 表达具体权限项。鉴权决策

流程在实现上以“策略加载—变量解析—动作匹配—结果裁决”为主线，并在读链路对召回候选进行二次过滤，保证检索与问答场景下不会返回越权结果。

在工程实现中，content_auth 的裁决遵循“动作优先”的原则：当请求为查看、编辑、删除、上下线等具体动作时，服务端先对目标内容加载其授权规则集合，并过滤出包含目标动作码的规则子集；随后按 auth_tactic 执行策略匹配。根据项目口径，auth_code 以精简集合表达动作权限，例如 `r/w/u` 分别对应“读/写/上下线”等能力，单条规则可携带多个动作码以减少规则数量。动作码与典型业务动作的映射如表 **??**所示。

表 4-3: content_auth 动作码映射示例

动作码	典型动作
r	内容可见、详情读取、检索/问答召回结果可返回
w	内容编辑、删除、类目/标签绑定等写操作
u	上线/下线、提审/审批相关的发布控制动作

在策略匹配层，平台支持免登录可见、所有外部用户可见、所有内部员工可见、员工匹配、部门匹配、角色匹配与自定义表达式匹配等策略。以部门匹配为例，系统将访问者的部门 ID 沿组织树向上回溯得到祖先链路集合，只要规则的 auth_variable 命中该集合即视为命中。策略裁决的关键实现片段如图 **??**所示。

为降低读链路开销，列表与检索场景尽量在查询阶段做粗过滤（例如按内容状态、空间/应用、引用集合），再对召回候选执行对象级授权校验，避免将复杂策略下推到检索引擎导致表达困难或泄露风险。

（3）审批与发布流程实现。为保证内容对外可用前的准确性与合规性，平台将发布动作纳入流程化审批控制：提审接口会创建审批记录并将内容状态置为待审批；审批通过后更新内容状态为已发布，同时产生内容发布事件驱动下游异步链路；审批驳回则回写拒绝原因并将内容退回可编辑状态。发布与下线接口会在保证幂等性的前提下触发事件，以驱动索引增量更新与统计回收，确保内容资产、索引与问答结果在发布前后保持一致。

审批服务以 content_approval 表承载审批单，核心字段包含 content_id、submitter_id、reviewer_id 与 status。提审时服务端校验内容必填字段、权限与当前状态，创建审批单并冻结内容进入待审批；审批通过与驳回均以审批单状态作为驱动依据，避免直接以内容状态作为唯一判定导致竞态。为避免重

```
public boolean checkContentAuth(long contentId, String actionCode, Visitor v) {
  List<AuthRule> rules = authDao.listRules(contentId);
  for (AuthRule r : rules) {
    if (!r.authCode.contains(actionCode)) continue;
    if (matchTactic(r.authTactic, r.authVariable, v)) return true;
  }
  return false;
}

private boolean matchTactic(String tactic, String var, Visitor v) {
  if ("public".equals(tactic)) return true;
  if ("internal".equals(tactic)) return v.isInternal;
  if ("employee".equals(tactic)) return String.valueOf(v.employeeId).equals(var);
  if ("department".equals(tactic)) return v.deptPath.contains(var);
  if ("role".equals(tactic)) return v.roleIds.contains(var);
  if ("custom".equals(tactic)) return exprEngine.eval(var, v.attrs);
  return false;
}
```

图 4-4: content_auth 策略匹配关键代码

复点击造成的重复发布，审批通过接口在更新审批状态前检查是否已处于 APPROVED，若已通过则直接返回并不重复触发事件，从而在“至少一次调用”语义下保持幂等。

审批与发布链路的实现重点在于两点：其一是以状态机约束操作合法性，避免非法状态流转；其二是发布后通过事件驱动触发下游异步处理，使“发布版本”在索引与数据侧尽快达成一致。关键实现片段如图 **??**所示。

4.2 内容应用模块的实现

内容应用模块面向最终用户与业务运营，提供内容展示、内容搜索与生成式问答等能力。与内容管理模块不同，内容应用链路强调高并发读性能、低延迟与可解释性：其一，所有对外可见结果必须以“发布版本”为输入口径；其二，检索与问答必须结合权限与范围配置做过滤，保证跨应用复用不引入越权风险；其三，生成式回答需提供证据引用，降低幻觉风险并支撑后续治理与

```
public void approve(ApproveReq req) {
  authService.checkPermission(req.auditor, req.spaceId, "
     CONTENT_APPROVE");
  Approval a = approvalDao.get(req.approvalId);
  if (a.isApproved()) return;
  if (!a.isPending()) {
    throw new BizException("invalid approval status");
  }
  approvalDao.pass(req.approvalId, req.comment);
  contentDao.updateStatus(a.getContentId(), "PUBLISHED");
  eventBus.publish(new ContentPublishedEvent(a.getContentId()
     , a.getSpaceId(), a.getAppId(), a.getId()));
}
```

图 4-5: 审批通过与发布事件触发关键代码

优化。

内容应用模块的关键服务端接口定义如表 **??**所示。

表 4-4: 内容应用模块接口表

服务名称	接口名称	功能描述
页面化展示	listContents / getDetail	基于空间/应用/类目/标签组织内容列表与详情页。
内容检索	search	多字段关键词检索与结构化筛选，结果受范围与权限控制。
智能问答	ask	RAG 多通道召回证据并生成回答，返回可追溯引用。
索引构建	onContentPublished	消费发布事件，完成切片与全文/向量索引增量更新。

参考师兄论文实现章节的组织方式，本节按“链路机制 → 关键实现片段 → 与流程图对应”的结构展开：首先说明页面化内容应用的范围确定与读口径，其次介绍多形态内容前置处理与索引构建，再说明 RAG 多通道召回与重排机制，最后给出生成式问答链路的实现与引用映射方式。

4.2.1 页面化内容应用机制

页面化内容应用机制用于支撑内容门户与管理后台的通用展示场景，包括目录、列表与详情页等典型页面形态。其实现目标是将“空间/应用边界、组织

方式（类目/标签）、发布口径、权限过滤”在读链路中统一起来，从而在不同入口下保持一致的可见范围与展示效果。

（1）范围确定与请求规约。页面化展示请求通常携带 space_id、app_id、category_id、tag 等参数。服务端在接入层首先完成身份校验，并根据调用方来源（门户/后台）与应用配置确定内容范围：以 space_id 与 app_id 作为强边界过滤条件，并在应用启用“引用内容”时将引用关系作为可见集合约束，从而限定仅展示被引用且已发布的内容。

（2）发布口径读取与一致性约束。展示链路统一以发布口径为准：列表页仅读取内容元数据与必要摘要；详情页在校验权限后读取发布版本正文。该策略保证展示、检索与问答链路对外一致，避免草稿内容在门户侧被提前可见。

（3）权限过滤与越权防护。展示链路在返回结果前执行权限过滤：先做 RBAC 的模块/操作级校验，再按 content_auth 策略对具体内容进行对象级判断。对于列表页，服务端在查询阶段尽量将权限条件下推；对于详情页，服务端在读取正文前进行强校验，避免越权内容被渲染或被前端缓存。

（4）关键实现片段。页面化展示的关键在于“范围确定 → 权限过滤 → 发布口径读取”的顺序约束。其核心实现片段如图 **??**所示。

4.2.2 内容检索服务实现

内容检索服务为门户与管理后台提供统一的关键词检索能力，并作为智能问答链路的基础召回通道之一。实现目标包括：多字段匹配（标题、摘要、正文等）、结构化过滤（类目、标签、内容类型）、分页与高亮展示，以及在空间/应用边界与权限约束下保证结果可控。

（1）索引字段与映射设计。检索索引以切片为基本文档粒度，并携带用于过滤与聚合的元信息字段：content_id、space_id、app_id、content_type、category_id、tag_ids、modify_time 等。文本字段按业务语义拆分为 title、summary 与 text，分别配置不同权重：title 用于精准命中提升排序稳定性，summary 用于概览召回，text 用于正文覆盖。对于 FAQ 类内容，question 与 answer 可分别入索引并设置差异化权重，以支持“只检索问题”或“问题 + 答案一起检索”等场景配置。

（2）查询构造与排序策略。查询侧采用“过滤优先、匹配其次”的策略：先用 term/range 过滤固定边界（space_id、app_id、状态为已发布、引用集合约束

```
public Page<ContentItemDTO> listContents(ListReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  authService.checkPermission(req.operator, req.appId, "
      CONTENT_READ");
  return contentQueryDao.queryPublished(scope, req.categoryId
      , req.tags, req.page, req.size);
}

public ContentDetailDTO getDetail(DetailReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  Content c = contentDao.get(req.contentId);
  authService.checkContentVisible(req.operator, scope, c.id);
  if ("knowledge".equals(c.type)) {
    String body = knowledgeDao.getBody(c.id);
    return renderService.renderKnowledge(c, body);
  } else if ("faq".equals(c.type)) {
    Faq f = faqDao.get(c.id);
    return renderService.renderFaq(c, f.question, f.answer);
  } else if ("course".equals(c.type)) {
    List<Lesson> lessons = courseDao.listLessons(c.id);
    return renderService.renderCourse(c, lessons);
  } else if ("case".equals(c.type)) {
    String detailJson = caseDao.getDetail(c.id);
    return renderService.renderCase(c, detailJson);
  }
  throw new BizException("unsupported content type");
}
```

图 4-6: 页面化内容展示关键代码

等），再对文本字段做 multi_match；排序以 BM25 为主，并叠加标题命中、更新时间、新鲜度等轻量特征。检索结果在返回前进行对象级权限校验，保证越权切片不会被返回。

（3）高亮与摘要生成。为提升可读性，检索服务对 title 与 text 开启高亮，并在无法高亮时退化为返回 summary 或对 text 做截断摘要。对于长正文切片，摘要以“命中片段前后窗口”形式生成，避免在列表页返回过长内容导致传输与渲染开销增大。

检索查询构造的关键实现片段如图 **??**所示。

```
public SearchResp search(SearchReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  authService.checkPermission(req.operator, req.appId, "
      CONTENT_READ");

  EsQuery q = new EsQuery();
  q.filterTerm("space_id", scope.spaceId);
  q.filterTerm("app_id", scope.appId);
  q.filterTerm("status", "PUBLISHED");
  q.filterTerms("content_id", scope.allowedContentIds); // 启
      用引用时的集合约束
  q.filterTerms("content_type", req.types);
  q.filterTerms("category_id", req.categoryIds);
  q.filterTerms("tag_ids", req.tagIds);
  q.multiMatch(req.query, fields("title^4", "summary^2", "
      text"));
  q.highlight("title", "text");
  q.sortByScoreThen("modify_time", "desc");

  List<SliceHit> hits = esClient.search(q, req.page, req.size
      );
  return authFilter.filterVisible(hits, req.operator); //
      content_auth 对象级过滤
}
```

图 4-7: 内容检索关键代码

4.2.3 多形态内容前置处理与索引构建

为支持高效检索与生成式问答，平台在内容发布后对多形态内容进行前置处理，并构建全文索引与向量索引。该链路由内容发布事件驱动：内容管理模块在审批通过后触发 ContentPublished 事件，索引构建服务消费事件并完成文档化、切片与索引写入。

（1）多形态前置处理与切片策略。不同内容形态在索引侧的目标是统一为可检索的最小证据单元（切片），并保证切片本身包含足够上下文信息以便被独立理解。根据项目口径，切片策略如下：

1. 文章（knowledge）：以固定长度为基础切片，并尽量保持段落语义完整性，避免跨段切分导致上下文断裂；切片携带文章标题、摘要、类目、标签等元数据。2. 问答（faq）：问题与答案作为完整语义单元不做切片处理，可分别入库并建立索引；召回阶段按场景组合使用（例如仅对 question 检索或对 question+answer 联合检索）。3. 课程（course）：考虑到成本，一个课节作为

一个切片；实现中通过大模型提取课节字幕或要点文本作为课节内容，以保证索引侧为可检索的文本。4. 案例（case）：每个案例作为一个切片；切片携带案例标题、摘要、类目、标签等元数据。

此外，图片与表格类内容在切片时需要特殊处理：图片使用多模态 embedding 模型实现“图文同空间”检索，并通过大模型生成文本化理解结果与图片链接一起入库；表格避免纯文本切分，按行保持语义完整并携带表头信息，以提升表格类知识在问答中的可用性。前置处理阶段对富文本标记、无意义噪声进行清洗，并保留必要结构信息以便后续组织引用。

在切片结构上，平台将每个切片建模为包含 `slice_id`、`content_id`、`space_id`、`app_id`、`content_type`、`text` 等字段的最小证据单元，并可选携带 `position` 用于表示在正文中的定位信息（例如段落序号或课节序号）。该设计使得召回结果既能用于生成式问答的证据输入，也能在输出侧映射为“内容标题 + 片段摘要 + 定位信息”的引用，降低用户在溯源时的跳转成本。

为保证切片可稳定更新，slice_id 采用可重复生成的规则：以 `content_id` 为前缀，拼接切片类型与 position 构成业务唯一键，例如 `<content_id>#P_<para_no>`、`<content_id>#L_<lesson_no>` 或 `<content_id>#T_<row_no>`。当内容发布触发索引增量更新时，同一 content_id 的切片可以按 slice_id 覆盖写入，从而在无需全量重建索引的前提下实现稳定更新。

切片生成的工程实现可以拆分为“内容提取 → 文本规范化 → 按类型切片 → 补充元信息”四步。内容提取阶段按 content_type 从对应子表或文档库加载正文或结构字段；文本规范化阶段对 HTML 标签、富文本标记、空白与不可见字符做清洗，并统一标点与换行以减少 embedding 噪声；按类型切片阶段根据不同形态选择不同的语义单元；最后补充元信息阶段将标题、摘要、类目、标签、权限等字段写入切片结构，便于召回与过滤。切片构建的关键实现片段如图 **??**所示。

图片与表格属于需要特殊处理的信息载体。图片切片的核心在于“可检索化”：通过多模态 embedding 模型将图片与文本映射到同一向量空间，并通过大模型生成 caption/要点作为文本化理解结果参与索引；切片同时携带图片 URL 与必要上下文信息，便于在回答中返回可点击溯源。表格切片的核心在于“保留结构语义”：按行切片并在每行切片中拼接表头字段，使得单行在脱离原表格时仍可被理解；对于列值较长的表格，按列优先级截断或做摘要，避免单切片超

```
public List<Slice> buildSlices(Content c) {
  if ("knowledge".equals(c.type)) {
    String body = normalize(knowledgeDao.getBody(c.id));
    return splitByParagraphOrLen(c, body); // 段落优先，必要时按长度切分
  } else if ("faq".equals(c.type)) {
    Faq f = faqDao.get(c.id);
    return buildFaqSlices(c, f.question, f.answer); // question/answer 两个语义单元
  } else if ("course".equals(c.type)) {
    List<Lesson> lessons = courseDao.listLessons(c.id);
    return buildLessonSlices(c, lessons); // 一课节一切片，字幕由LLM提取
  } else if ("case".equals(c.type)) {
    String json = caseDao.getDetail(c.id);
    String text = normalize(caseExtractor.toText(json)); // 字段抽取与拼接
    return listOf(slice(c, "C_0", text));
  } else if ("image".equals(c.type)) {
    ImageMeta m = imageDao.get(c.id);
    String caption = llmClient.caption(m.url);
    return listOf(slice(c, "I_0", caption + " " + m.url));
  } else if ("table".equals(c.type)) {
    Table t = tableDao.get(c.id);
    return buildTableRowSlices(c, t.header, t.rows); // 每行拼接表头，保证行语义完整
  }
  return emptyList();
}
```

图 4-8: 切片构建关键代码

出上下文窗口。

（2）元信息绑定与权限下推。切片索引时携带 content_id、modify_time、space_id、app_id 与 content_auth 相关元信息，用于召回阶段的范围确定与权限过滤。通过元信息绑定，可以保证任意召回结果都能回溯到具体内容及其发布时间点，并在服务侧做对象级可见性判断。索引同时构建 ES 与 embedding 两套索引：ES 侧侧重字面匹配与可解释高亮，embedding 侧侧重语义相似与长尾覆盖，两条通道互补并为后续 RAG 提供更稳健的候选集合。

索引侧采用 `content_id + modify_time` 作为发布口径标识：当内容发布后触发索引更新事件，索引构建服务拉取当前已发布内容及其更新时间戳，作为索引写入的幂等与覆盖依据。召回阶段以 space_id、app_id 与内容状态作为粗过滤条件，并在服务侧对候选内容执行 content_auth 对象级校验，避

免复杂授权策略直接下推到索引侧导致表达困难。

（3）增量更新与幂等控制。索引构建消费发布事件后进行增量更新：同一 content_id 的旧索引会被覆盖或标记过期，查询侧只命中最新发布口径。为避免重复消费导致的脏写，索引构建服务以 (content_id, modify_time, index_type) 作为幂等键，并在写入前检查已处理记录，从而在“至少一次投递”语义下实现最终一致。

索引构建在工程实现上通常以 `content_id` 为主键进行覆盖更新：全文索引将切片作为文档写入，并携带 content_id 用于聚合与删除；向量索引将 embedding 向量与 slice_id 绑定，并在更新时先按 content_id 删除旧切片或标记旧切片不可见，再写入新切片。为降低峰值写入压力，索引构建服务对批量发布采用分批处理，并对失败批次进行重试；在重试过程中仍以幂等键约束写入，避免重复写入造成索引膨胀。

（4）关键实现片段。索引构建核心在于“事件消费 → 拉取发布版本 → 切片 → 双索引写入”。其关键实现片段如图 **??**所示。

```
public void onContentPublished(ContentPublishedEvent e) {
  Content c = contentDao.get(e.contentId);
  if (!c.isPublished()) return;
  if (indexMetaDao.exists(e.contentId, c.modifyTime, ”
     FULLTEXT”)) return;
  List<Slice> slices = sliceService.buildSlices(c);
  esClient.upsertDocuments(c.spaceId, c.appId, c.id, c.
     modifyTime, slices);
  vectorClient.upsertEmbeddings(c.spaceId, c.appId, c.id, c.
     modifyTime, slices);
  indexMetaDao.markDone(e.contentId, c.modifyTime);
}
```

图 4–9: 索引构建关键代码

4.2.4 基于 RAG 的多通道召回与重排

基于 RAG 的多通道召回与重排用于在生成式问答场景下提供高质量证据集合，并在可控成本下兼顾召回率与精度。实现层面，平台采用“多通道召回—融合去重—重排—证据组织”的流水线结构：全文通道负责精准关键词召回，向量通道负责语义相似召回，两者融合后进行去重与重排，最终输出与问题最相关的 slice 集合。

整体链路包含“问题改写 → 多通道召回 → 融合去重 → 重排 → 证据组织”五个关键步骤。问题改写位于召回之前：系统将用户输入改写为更标准化的表达，并结合词典归一化与停用词处理生成用于检索的 query，从而提升在同义改写、口语化表达与长尾问法场景下的召回稳定性；随后分别在 ES 与向量索引上执行 TopK 召回并施加 MinScore 阈值过滤；融合阶段对两路候选去重并合并召回来源信息；重排阶段综合召回分数与多特征输出 TopN 证据；最后将证据按 content 聚合并组织为适配上下文窗口的引用片段集合，供后续生成回答使用。

（1）多通道召回。全文召回基于 ES 的 BM25 与字段权重，支持标题、摘要与正文多字段匹配；向量召回基于 embedding 相似度检索，覆盖同义改写与长尾问法。为控制延迟，系统对两路召回设置 topK 上限，并在召回阶段尽量下推范围与权限过滤条件，减少后续候选规模。

为降低长尾问法与口语化输入对召回的影响，平台在召回前引入“问题改写+query 规整”的预处理步骤：问题改写将用户输入改写为更标准化的表达以提升召回质量；query 规整则对改写后的文本进行分词与停用词处理，并对包含业务术语与缩写的词典进行归一化。对于低质量输入（过短、纯符号、无有效词），系统直接触发未命中返回并进入覆盖度候选。预处理后的 query 同时用于全文检索与向量检索，以保证多通道召回输入一致。

问题改写在实现上以提示词约束输出结构，避免模型自由发挥导致关键信息丢失。提示词要求模型在结合对话上下文的前提下抽取“对象、场景、诉求”三要素，并输出“完整表述”和更接近知识标题的“主要问题”，用于后续召回。问题改写提示词模板如图**??**所示。

召回阶段的关键参数包括 TopK 与 MinScore：TopK 用于控制返回候选切片数量，MinScore 用于设置最小相似度阈值以过滤低相关候选。表**??**给出了关键参数的工程含义。

（2）融合去重与候选裁剪。多通道返回的候选以 slice_id 为主键融合去重，并对同一 content_id 的多个候选进行聚合，避免证据过度集中在单一内容上。融合阶段会根据召回来源与初排分数进行粗排裁剪，为后续重排降低成本。

（3）重排与证据组织。重排阶段以多特征排序，包括语义相似分、关键词覆盖、标题命中、内容新鲜度等，并对候选进行多样性约束。证据组织阶段将 topN 的 slice 按文档聚合，并截断拼接为适合大模型上下文窗口的证据段

```
# 角色
你是商业化业务知识助手，负责将用户问题改写为更易检索的表达。

# 输入
<context>
{history_messages}
</context>
<query>
{raw_query}
</query>

# 定义
对象：问题围绕的主体/概念（名词为主）
场景：问题发生的背景/条件/状态（可包含多个限定）
诉求：用户希望获取的信息或完成的动作（是什么/为什么/怎么办
    等）

# 任务
1）结合上下文抽取对象/场景/诉求三要素，若缺失可用上下文补全
2）若query包含错别字或拼音，先修正再改写
3）严格基于query与上下文，禁止引入未提及的新要素与操作
4）生成两种表达：
   - 完整表述：必须包含对象/场景/诉求，用于问答理解
   - 主要问题：更接近知识库标题，更书面精炼，用于召回
5）字数限制：每行不超过30字；句尾不使用任何标点
6）输出格式必须严格为两行

# 输出格式
【完整表述】：{full_query}
【主要问题】：{main_question}
```

图 4-10: 问题改写提示词模板

表 4-5: RAG 召回关键参数

参数	含义
TopK	单通道召回的候选上限，用于控制延迟与后续重排开销
MinScore	最小相似度阈值，低于阈值的候选直接丢弃以提升精度与可解释性
TopN	重排后最终用于证据组织的切片数量，受上下文窗口与引用展示限制

落，同时保留每段证据的来源标识，用于输出引用。

重排实现采用“粗排 + 精排”的两阶段策略：粗排阶段以召回通道得分与轻量特征（标题命中、关键词覆盖）进行线性加权，快速裁剪候选集合；精排阶段对裁剪后的候选计算更细粒度特征，例如 query 与 slice 的语义相似度、与标题/摘要的覆盖度、以及同一 content 的多样性约束，并输出最终 topN 证据。重排的关键实现片段如图 **??**所示。

```
public List<SliceHit> rank(String query, List<SliceHit> hits,
    int topn) {
  for (SliceHit h : hits) {
    double score = 0;
    score += 0.5 * h.recallScore;
    score += 0.3 * h.keywordCover;
    score += 0.2 * h.titleHit;
    h.finalScore = score;
  }
  List<SliceHit> sorted = sortByScore(hits);
  return diversifyByContent(sorted, topn);
}
```

图 4-11: 重排关键代码

（4）关键实现片段。RAG 召回与重排的关键实现片段如图 **??**所示。

```
public EvidencePack retrieveEvidence(AskReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  authService.checkPermission(req.operator, req.appId, "
      QA_READ");
  String rewritten = rewriteService.rewrite(req.query);
  String q = queryNorm.normalize(rewritten);
  List<SliceHit> esHits = esClient.search(q, scope, req.
      topkEs, req.minScoreEs);
  List<SliceHit> vecHits = vectorClient.search(q, scope, req.
      topkVec, req.minScoreVec);
  List<SliceHit> merged = mergeService.dedupAndMerge(esHits,
      vecHits);
  List<SliceHit> reranked = rerankService.rank(q, merged, req
      .topn);
  return evidenceBuilder.build(reranked);
}
```

图 4-12: RAG 召回与重排关键代码

4.2.5 生成式问答流程实现

生成式问答流程将 RAG 检索得到的证据集合注入提示词中生成回答，并在输出侧附带引用信息，形成“问答—证据—治理”的闭环入口。实现目标是：在保证回答准确性的前提下，降低幻觉风险，并通过引用定位到具体内容片段以支撑后续治理。

（1）Prompt 构造与上下文控制。系统在构造提示词时将用户问题、业务约束与证据集合分段拼接，并对证据长度进行裁剪以适配上下文窗口。为保证引用可映射，系统在证据段落中附带 slice_id 与标题信息，使引用能够回溯到具体 slice。

提示词结构在实现上通常包含三部分：系统约束（禁止编造、必须基于证据回答）、任务描述（回答风格、输出结构）、证据集合（按 slice 聚合并编号）。证据集合为每条证据分配 key（可使用 S1/S2 等序号，也可直接使用 slice_id），并要求模型在回答中以固定引用标记声明知识来源，例如 `[引用]<key1,key2>`。该设计降低了引用解析难度，也使得在回答失败时可以退化为返回证据摘要而不丢失溯源能力。生成回答提示词模板如图 **??**所示。

（2）回答生成与引用解析。模型生成结果返回后，系统对回答文本中的引用标识进行解析，并转换为可展示的“内容标题 + 片段摘要 + 跳转链接”。当模型未给出引用或引用解析失败时，系统退化为按证据集合的 topN 进行引用补齐，保证每次回答都有最小可追溯集合。

引用映射在实现上以 slice_id 为桥梁：citationParser 将模型输出的引用编号映射到证据集合中的 slice_id，并进一步根据 slice_id 回溯到 content_id 与可选的定位字段（课节序号、段落序号等），最终生成前端可展示的引用列表。为避免模型输出无效编号或重复编号，解析器对引用编号做合法性校验与去重，并限制最大引用数量，保证展示区域稳定。

（3）异常退化与安全控制。当检索阶段未召回到足够证据时，系统返回“未命中/建议补充内容”的提示，并将该问题作为覆盖度治理的候选输入；当模型生成超时或失败时，系统退化为检索结果摘要返回，保证服务可用性。读链路全过程保留权限过滤与范围约束，避免越权证据进入提示词。

（4）关键实现片段。生成式问答的关键实现片段如图 **??**所示。

```
# system prompt (excerpt)
你是商业化业务知识问答专家，擅长解答广告/电商/商业化相关问题。
回答必须遵守:
1) 必须基于参考资料作答，禁止编造数据、结论与规则，禁止出现矛盾结论
2) 禁止偏袒篇幅较长或位置靠前的资料，需要综合判断相关性
3) 答案使用中文，markdown输出，结构清晰，字数不超过500字
4) 若答案使用了任意参考资料的知识点，必须在相关位置插入引用标记:
   [引用]<key1,key2,...>  (key来自证据集合中的ID或序号，逗号分隔)
5) 禁止出现“文档/参考资料/根据xxx”等暴露资料来源的字眼
6) 参考资料可能包含图片占位: ![desc](http...)，若使用图片信息可原样输出该图片
7) 若参考资料不足以回答，给出保守回答并提示进一步确认或补充信息

# user prompt (template)
禁止创造公式!
<question>
{raw_query}
{rewritten_query}
</question>
<doc>
{evidence_documents_with_keys}
</doc>
```

图 4-13: 生成回答提示词模板

```
public AskResp ask(AskReq req) {
  EvidencePack pack = retrieveEvidence(req);
  if (pack.isEmpty()) {
    coverService.recordMiss(req);
    return AskResp.miss("未命中可用内容，建议补充知识");
  }
  String prompt = promptBuilder.build(req.query, pack);
  String answer = llmClient.generate(prompt);
  List<Citation> cites = citationParser.parseOrFallback(answer, pack);
  return AskResp.ok(answer, cites);
}
```

图 4-14: 生成式问答关键代码

4.3 内容治理模块的实现

内容治理模块面向内容规模化带来的质量问题，构建“自动检测—任务化分发—人工处理—复检回收”的闭环机制。实现上，平台以重复度、歧义度与覆盖度三类问题为核心，将离线分析结果转化为可流转的线上治理任务，并与内容管理的修订与发布流程打通，保证治理处理后可触发索引更新与指标回收。

内容治理模块的关键服务端接口定义如表 **??**所示。

表 4-6: 内容治理模块接口表

服务名称	接口名称	功能描述
任务管理	createMainTask / getMainTaskDetail	创建治理主任务并管理计算状态与配置。
检测计算	runDuplicateAmb / runCoverage	执行重复/歧义检测与覆盖度检测，产出任务候选。
任务流转	claimTask / submitResult / markDone	指派或认领任务，提交处理结论并更新状态。
复检回收	recheck / autoClose	对任务范围周期复检，自动关闭已消除问题。

治理任务线上化需要在存储层固化“主任务-子任务”的任务模型。主任务负责一次治理计算的配置与状态管理，子任务承载具体问题内容与处理结论。根据项目手册口径，治理任务相关表的核心字段如表 **??**所示。

表 4-7: 治理任务数据表（核心字段）

表名	关键字段（示例）	说明
govern_main_task	id, name, space_id, session_period, recall_threshold, status	治理主任务，承载会话周期、阈值等参数与计算状态。
govern_dup_amb_task	id, main_task_id, app_id, task_type, content_id_a, content_id_b, status, assignee_id, result	重复/歧义子任务，描述内容对、任务类型与处理结果。
govern_coverage_task	id, main_task_id, app_id, item_type, item_value, score, status, assignee_id	覆盖度子任务，描述覆盖对象（类目/标签/概念/定义/query 等）与覆盖评分。

在线治理闭环包含“计算生成 → 人工处理 → 系统巡检回收”三类动作：主

任务创建后进入计算中状态，计算完成后生成子任务列表；业务人员在任务页面对重复/歧义任务标记处理结果（下线/保留/合并等），对覆盖任务标记是否需要补充内容并完成处理；系统侧每天定时巡检自上次巡检以来发生内容变更、上线、下线或删除的内容，复用 LLM 判别与召回评估对关联任务进行复检，若问题已消除则自动将子任务标记为完成，从而避免任务长期堆积造成统计失真。

4.3.1 重复度与歧义度检测实现

重复度与歧义度检测采用“候选召回 + 模型判别”的方式避免全量两两比较。首先以向量检索召回每条内容的 TopK 相似候选，形成候选对集合；随后对候选对构造判别提示词，使用大模型输出相似度与歧义度判断；最后将判别结果聚合为重复对与歧义对，并输出片段级定位信息供人工复核与任务化流转。

重复度治理在业务侧需要可解释的分级口径，以便运营判断“是否需要合并/下线/保留”。平台按重复占比对重复程度进行分级，典型口径如表 **??**所示。

表 4-8: 重复度分级口径示例

分级	重复占比	说明
完全重复	100%	两篇内容几乎完全一致，包括用词与顺序。
极度重复	80%–100%	主体相同，仅少量差异。
中度重复	50%–80%	主体相同，部分描述差异明显。
轻微重复	25%–50%	有一定重叠但存在较多差异。
不重复	< 25%	主体不同或重叠很少。

（1）候选召回与降本。候选召回阶段仅在同一空间或同一应用范围内生成候选对，并对相似度设置阈值下限以控制候选规模。该策略能够将检测复杂度从 $O(n^2)$ 降低到 $O(nk)$，更适合在内容规模增长时持续运行。

候选召回的关键在于确定“相似度比较的语义单元”。对于 knowledge 等长文内容，平台优先以切片为单位进行召回：先为每条内容聚合其切片向量（例如取 topN 切片均值或取标题/摘要向量），在内容粒度产生候选对；随后在判别阶段再定位到具体冲突片段。对于 faq，候选召回以标准问题与相似问集合为主，使得不同问法能够被聚合到同一知识单元下，降低重复维护的概率。

（2）判别提示词与结构化输出。判别阶段将“内容 A+ 内容 B+ 上下文约束”构造成统一的判别输入，并要求模型输出结构化字段（是否重复、是否歧义、冲突点摘要、建议处理方式等），便于后续落表与任务创建。

判别提示词在实现上强调两点：其一是仅基于提供的内容进行判断，禁止引入外部知识；其二是输出结构化 JSON，包含布尔判断与可解释字段，便于程序解析并落表。判别输出通常包含 `duplicate/ambiguous` 两类标记、冲突摘要、建议处理动作（合并/保留/下线/补充澄清）以及可选的冲突片段定位。判别流程的关键实现片段如图 **??**所示。

重复度与歧义度的判别提示词需要在“可解释”与“可解析”之间取得平衡：既要提供足够的对比信息与裁决规则以降低误判，也要保证输出为机器可解析的结构化 JSON，便于任务生成与后续复检。判别提示词模板如图 **??**所示。

（3）结果聚合与任务生成。检测结果按内容对聚合，并与内容元信息、引用范围等字段拼装形成可流转的治理子任务。重复/歧义检测的关键实现片段如图 **??**所示。

4.3.2 覆盖度检测实现

覆盖度治理用于持续发现“缺知识”与薄弱领域，并将缺口转化为可闭环的补充任务。实现中覆盖度检测主要基于会话数据：在设定会话周期与召回阈值的前提下，对用户原声执行召回评估，召回结果未满足阈值要求的视为缺知识；对缺知识原声进行聚类生成覆盖任务，由业务人员补充内容或完善现有内容，并通过周期复检对任务范围进行回收。

覆盖度检测在实现上通常包含两条互补路径：其一是知识体系驱动，对接广告与商业化知识体系（类目、标签、概念、定义等）作为覆盖对象集合，逐项执行 RAG 召回并用大模型判断“是否覆盖”并打分，得到覆盖度分布；其二是需求驱动，从用户 query 与未命中行为中抽取真实需求，按语义聚类形成覆盖任务并推动内容补充。本项目实现以需求驱动路径为主，并在主任务配置中保留召回阈值与会话周期等参数，便于与知识体系驱动结果在同一指标口径下对齐。

覆盖度判断在工程实现上同样采用“召回 + 判别”的模式：先对覆盖对象（类目/概念/定义/query 等）执行 RAG 召回得到 TopK 候选证据，再由大模型判断候选证据是否能够覆盖该覆盖对象，并输出覆盖结论与可解释字段用于落表。覆盖度判别提示词模板如图 **??**所示。

```
# 重复度判别（template）
你的任务是判断两篇内容的重复程度，分级为：完全重复/极度重复/中度重复/轻微重复/不重复。
<title1>{title1}</title1>
<content1>{content1}</content1>
<title2>{title2}</title2>
<content2>{content2}</content2>
请按步骤判断：
1) 参考标题提炼两篇内容的主体对象与描述点
2) 判断主体是否一致；若一致，逐一对比描述点并估算重复占比（取占比更高的一侧）
3) 按分级标准输出最终结论
输出必须为JSON（不要输出其他说明）：
{"analysis":"主体与描述点对比、重复占比依据","decision":"重复度分级"}

# 歧义度判别（template）
你的任务是判断两组内容在“相同主体”下的描述是否存在冲突。
<business1>{business1}</business1>
<title1>{title1}</title1>
<content1>{content1}</content1>
<business2>{business2}</business2>
<title2>{title2}</title2>
<content2>{content2}</content2>
规则：
1) 先识别主体并判断是否为同一主体（需结合业务场景、限定词、活动/行业等，不可模糊联想）
2) 若无相同主体，直接判定为一致
3) 若主体相同，仅基于文本中“直接、明确、无歧义”的事实判断是否冲突，禁止推断
4) 冲突示例：支持/不支持、时间点或覆盖范围不一致、数值口径或公式组成不一致
5) 示例/URL/举例可忽略，不作为冲突依据
输出必须为JSON（不要输出其他说明）：
{"entity":"存在不一致的主体(无则为空)","consistency":"一致/不一致",
 "conflicts":[{"description_A":"不一致描述A","description_B":"不一致描述B"}],
 "conflict_summary":"判定依据与命中的规则"}
```

图 4-15: 重复度与歧义度判别提示词模板

（1）缺知识识别。系统将每条 query 按空间/应用范围进行召回评估，若 top1 相似度低于阈值或无有效证据，则将该 query 记录为缺知识候选。

缺知识候选的来源既包括用户在智能问答中的原始 query，也包括搜索框的查询词与未命中行为。实现中平台对缺知识候选进行基础清洗：去除纯噪声 query、合并大小写与数字格式差异、过滤过短文本，并按空间/应用聚合以避

```
public LlmJudgeResp dupAmbJudge(Long a, Long b) {
  String contentA = contentLoader.loadPlainText(a);
  String contentB = contentLoader.loadPlainText(b);
  String prompt = promptTpl.render(contentA, contentB);
  String json = llmClient.generate(prompt);
  return jsonParser.parse(json);
}
```

图 4-16: 重复度与歧义度判别关键代码

```
public void runDupAmb(MainTask task) {
  for (Long contentId : taskScopeDao.listContents(task.
      spaceId)) {
    List<Long> candidates = vectorClient.topKSimilar(
        contentId, task.recallThreshold, task.topk);
    for (Long otherId : candidates) {
      LlmJudgeResp r = llmJudge.dupAmbJudge(contentId,
          otherId);
      if (r.isDuplicate() || r.isAmbiguous()) {
        dupAmbTaskDao.upsert(task.id, task.appId, r.toTask(
            contentId, otherId));
      }
    }
  }
}
```

图 4-17: 重复度与歧义度检测关键代码

免跨边界混淆。召回评估同时参考全文与向量两路结果，当两路均未达到阈值要求时才判定为缺知识，以降低单通道失误带来的误判。

（2）覆盖任务生成。覆盖度治理的目标是将“缺知识候选”转化为可闭环的补充任务。在工程实现中，query 的语义聚类属于数据洞察模块会话分析的前置步骤：洞察侧将历史 query 聚类并产出聚类指标宽表，治理侧复用该聚类结果进行任务生成与分发。

具体而言，平台从洞察侧筛选低解决率或高转人工的聚类主题，结合召回评估结果判定为覆盖薄弱点，并生成覆盖任务。覆盖任务以 `cluster_id` 或代表性 query 作为定位依据：一方面，cluster_id 用于在洞察侧下钻到明细会话与原始 query 集合；另一方面，代表性 query 用于在治理侧做复检回收。该设计将“发现问题（洞察）”与“解决问题（治理）”解耦，使覆盖度治理能够持续从真实用户需求中获得输入而不重复建设聚类能力。

（3）复检回收。任务处理后，系统对原聚类代表 query 进行复检，若已能

```
# 角色
你是内容覆盖度评估专家，负责判断“目标知识点”是否被证据内容覆盖。

# 输入
<item>
{item_type}: {item_value}
</item>
<query>
{represent_query}
</query>
<docs>
{topk_evidence_with_keys_and_text}
</docs>

# 规则
1) 仅基于docs判断，禁止引入外部知识与推断
2) 若docs中能直接回答/定义/说明该item，则判定为覆盖
3) 若仅弱相关或无法支撑结论，则判定为未覆盖
4) 输出必须为JSON，便于程序解析落表

# 输出(JSON)
{
  "covered": true/false,
  "score": 0.0-1.0,
  "evidence": ["key1","key2"],
  "reason": "覆盖/不覆盖的简要依据",
  "suggestion": "若未覆盖，给出应补充的要点"
}
```

图 4-18: 覆盖度判别提示词模板

召回到满足阈值的证据则自动完成任务，避免“内容已补充但任务未更新”导致统计失真。

复检回收以任务中沉淀的代表性 query 为输入，复用问答召回能力对其进行周期性评估；当 top1 证据满足阈值且引用内容处于已发布状态时，任务自动进入完成状态。对于重复/歧义类任务，复检回收则以内容对为输入，若内容被合并、下线或修订后不再判定为重复/歧义，则任务自动关闭，从而实现“治理处理 → 复检验证 → 任务闭环”的持续运营机制。

4.3.3 治理任务线上化与闭环机制

治理任务线上化将检测结果以主任务与子任务两层结构落地。主任务负责一次治理计算的配置与状态管理，包含任务名称、空间、会话周期与召回阈值

等上下文信息；子任务承载具体问题与处理结论。重复/歧义任务支持人工判断下线、保留或合并；覆盖任务支持标记“无需补充/需要补充”并填写说明，处理引发内容修订后可进入内容管理的提审与发布流程，发布后由索引构建服务消费发布事件完成增量索引更新。

在任务状态设计上，主任务的状态通常包括“新建/计算中/计算完成”，子任务的状态包括“待处理/处理中/已完成/已关闭”。平台将任务处理与内容发布链路打通：当责任人根据覆盖任务补充内容或根据重复任务合并下线内容后，内容进入提审与审批发布；发布事件触发索引更新后，复检回收再对任务范围进行验证并更新任务状态。该设计将治理从一次性离线分析转化为可持续的线上闭环，使治理效果能够稳定反哺内容应用与数据洞察。

（1）主任务创建与计算执行。主任务创建由空间维度入口发起：用户填写任务名称、选择业务空间、设置会话周期与召回阈值，提交后主任务进入“计算中”。计算任务由后台异步执行，避免在前台请求中长时间阻塞。实现上主任务包含两类计算：重复/歧义任务计算以空间内已发布内容为输入，按 TopK 召回生成候选对并调用 LLM 判别；覆盖任务计算以洞察侧产出的会话聚类为输入，筛选低解决率主题并结合召回评估生成覆盖子任务。计算完成后主任务状态更新为“计算完成”，并生成子任务列表供责任人处理。

为保证计算可重复触发且不产生重复任务，子任务写入采用幂等 upsert：重复/歧义子任务以 (main_task_id, task_type, content_id_a, content_id_b) 作为业务唯一键，覆盖子任务以 (main_task_id, app_id, item_type, item_value) 作为业务唯一键。主任务的计算过程以批处理方式推进，并周期性写入进度与耗时信息，便于前端展示计算状态与失败定位。

（2）任务分发与处理流程。子任务分发支持“指派”与“认领”两种方式：指派模式由空间负责人按业务域将任务分配到具体责任人；认领模式允许运营人员从任务池领取处理。重复/歧义任务的处理动作通常包括下线、保留与合并：下线用于消除重复入口，保留用于确认差异合理，合并用于将多个内容统一为权威内容并调整引用关系；覆盖任务的处理动作通常是补充内容或标记无需补充，并填写说明。任务处理会记录处理结果与处理人，并推动必要的内容修订进入“提审 → 审批 → 发布”链路。

任务状态机在实现上需要满足幂等与并发安全：同一子任务只允许一个责任人进入处理中状态，避免多方同时处理导致冲突；子任务完成后不可回退到待处理，除非被复检回收重新打开。表 **??**给出了主任务与子任务的状态口径

示例。

表 4-9: 治理任务状态口径示例

任务类型	状态	含义
主任务	NEW/RUNNING/FINISHED	新建、计算中、计算完成
子任务	TODO/DOING/DONE/CLOSED	待处理、处理中、已完成、已关闭（复检确认问题消除）

（3）系统巡检与复检回收。为避免“内容已变更但任务未更新”导致任务长期堆积，平台增加定时巡检：每天扫描自上次巡检以来发生上线、下线、删除或内容修订的内容，定位关联的重复/歧义子任务并重新执行判别；对覆盖子任务则复用召回评估对代表性 query 进行复检。若重复/歧义问题已消除或覆盖已达标，则系统自动将子任务标记为已关闭或已完成。

巡检实现的关键在于“只重算受影响范围”。平台以 content_id 为索引维护子任务的反向引用，内容变更时仅查出关联子任务进行重算，避免对全量任务重复计算。复检回收的关键实现片段如图 **??**所示。

```
public void dailyRecheck(Date since) {
  List<Long> changed = contentDao.
      listChangedPublishedContents(since);
  for (Long contentId : changed) {
    for (DupAmbTask t : dupAmbTaskDao.listByContent(contentId
        )) {
      LlmJudgeResp r = llmJudge.dupAmbJudge(t.contentIdA, t.
          contentIdB);
      if (!r.isDuplicate() && !r.isAmbiguous()) dupAmbTaskDao
          .close(t.id);
    }
  }
  for (CoverageTask t : coverageTaskDao.listOpenTasks()) {
    boolean ok = recallEvaluator.covered(t.representQuery, t.
        threshold);
    if (ok) coverageTaskDao.close(t.id);
  }
}
```

图 4-19: 治理任务复检回收关键代码

4.4 数据洞察模块的实现

数据洞察模块面向业务与运营提供“可观测 + 可分析 + 可追溯”的闭环能力。整体实现上采用“前端埋点与服务端事件采集 → 数仓统一加工与指标口径沉淀 → 洞察服务查询与权限控制 → 前端可视化与下钻明细”的链路。数据加工与口径沉淀由数仓侧负责，本项目侧重点在事件规范、采集链路、与数仓的对接与查询服务建设。

数据洞察模块的关键服务端接口定义如表 **??**所示。

表 4-10: 数据洞察模块接口表

服务名称	接口名称	功能描述
事件上报	reportEvent	统一事件模型上报与关键链路服务端埋点。
指标查询	queryPageMetrics / queryContentMetrics	按时间范围与维度查询页面/内容指标宽表。
会话分析	querySessionCluster / querySessionDetail	查询聚类指标并下钻会话明细。

4.4.1 内容应用数据采集机制

数据采集需要覆盖页面化消费链路与智能问答链路两类场景。在页面化链路中，采集曝光、点击、搜索输入与结果点击等行为数据；在问答链路中，采集问题输入、召回证据、生成结果与转人工等链路数据。平台对事件名与字段进行统一定义，例如 page_view、click、query_submit、rag_recall、llm_generate、handoff_intent 与 handoff_success 等，并统一携带 app_id、space_id、session_id、trace_id 与 content_id 等公共维度字段，便于端到端追踪与明细还原。

为保证事件口径可复用且便于扩展，平台将每条事件抽象为“公共维度 + 事件特有扩展字段”的统一结构：公共维度用于跨事件关联与链路追踪，扩展字段用于承载不同事件的特有信息（例如检索关键词、召回通道、候选数量、引用列表等）。事件明细的核心字段设计如表 **??**所示。

扩展字段 ext 用于承载不同事件的特有信息。以智能问答链路为例，rag_recall 事件会携带召回通道、TopK、MinScore、候选数量与最终证据集合摘要；llm_generate 事件会携带 prompt 长度、生成耗时、引用数量、是否触发退化等信息。表 **??**给出了典型事件的 ext 字段示例，用于支撑后续“链路分析 → 问题定位 → 策略优化”的闭环。

表 4-11: 事件明细核心字段设计

字段名	类型	说明
event_name	string	事件名（page_view/click/query_submit/rag_recall/llm_generate 等）
app_id / space_id	bigint	组织边界维度，用于范围过滤与聚合
user_id / visitor_id	bigint/string	用户标识（登录用户/匿名访客），用于 UV 与分群分析
session_id	string	会话标识，用于多轮对话与漏斗分析
trace_id	string	链路标识，用于端到端追踪与去重
content_id	bigint	内容标识，可为空（例如曝光/点击未关联内容时为空）
timestamp	datetime	事件发生时间
channel / device	string	渠道与设备维度，可为空
ext	json	扩展字段（query、召回通道、得分、引用列表等）

表 4-12: ext 字段示例（典型事件）

事件名	ext 字段示例
rag_recall	query, rewritten_query, channels(es/vec), topk, min_score, hit_cnt, evidence_ids
llm_generate	model, prompt_len, latency_ms, token_out, citation_cnt, degraded(bool)
handoff_intent	reason, confidence, stage(recall/generate)

采集链路上，前端埋点负责覆盖页面曝光、点击、搜索等用户侧行为；服务端埋点覆盖 RAG 召回、重排、生成与转人工等关键节点，保证链路关键事件不丢失。服务端在生成 trace_id 后将其贯穿写入召回与生成事件，使得洞察侧可以按 trace_id 还原单次问答的证据与决策链路。为避免重复上报导致的指标膨胀，平台在采集接入层对同一 (trace_id, event_name) 做幂等去重，并对异常流量进行限流保护，保证事件链路稳定。

4.4.2 会话分析与效果指标设计

在指标体系设计上，平台从页面、会话与内容三个视角承接数仓侧产出的聚合指标：页面监控关注 PV/UV/点击率；会话分析关注会话数量、模型识别解决率、意向转人工率与转人工率；内容分析关注内容被召回次数、被生成次数以及转人工相关指标。洞察服务提供按时间范围、空间/应用、渠道与页面等维度的只读查询接口，支持从聚合图表下钻到会话明细。

会话分析在实现上以 session_id 为主键组织多轮交互，并以 trace_id 串联单轮问答的召回与生成事件。聚合视图以“会话级”与“聚类级”两种口径输出：会话级指标用于观察整体趋势与转化漏斗；聚类级指标用于定位高频问题簇并下钻到原始 query 与证据。洞察侧通过聚类 ID 为每条会话打标，从而实现“主题簇 → 明细会话 → 引用内容”的端到端下钻路径。

会话聚类用于将大量历史 query 归并为有限的主题簇，便于从统计视角识别高频问题与薄弱点。聚类构建采用基于语义相似图的连通分量方法：将每条 query 向量化写入向量索引，设定最小相似度阈值 `min_score`，相似度 ≥ 阈值的 query 之间建立边；从未归类 query 出发递归扩展其邻居集合，形成一个连通分量作为聚类。聚类构建的关键实现片段如图 **??**所示。

会话聚类完成后需要生成可读的主题标题与关键词，便于在洞察看板与治理覆盖任务中定位问题簇。平台在实现中使用大模型对同一聚类的代表性 query 集合进行总结，输出摘要、关键词与代表 query 列表，并将摘要作为 cluster_title 写入聚类指标宽表。聚类主题总结提示词模板如图 **??**所示。

指标查询接口在实现上遵循只读原则：洞察服务不直接参与指标计算，仅对接数仓侧沉淀的宽表与明细表。为支撑高频查询，洞察服务对热点维度（例如近 7 天、近 30 天、单空间 TopN）使用缓存或物化视图降低延迟，并对查询接口增加时间范围与返回条数限制，避免明细下钻对后端与数仓造成过高压力。

4.5 本章小结

本章对内容管理、内容应用、内容治理与数据洞察四个模块的关键实现进行了说明。内容管理模块以统一内容模型与状态机约束实现多形态内容的规范化生产与发布口径控制，并通过引用与权限机制支撑跨应用复用与越权防护；内容应用模块在发布事件驱动下构建索引，提供页面化展示、检索与 RAG 问

```
public List<Cluster> buildClusters(List<QueryItem> items,
    double minScore) {
  Set<String> visited = new HashSet<>();
  List<Cluster> clusters = new ArrayList<>();
  for (QueryItem q : items) {
    if (visited.contains(q.id)) continue;
    Queue<String> queue = new ArrayDeque<>();
    Cluster c = new Cluster();
    queue.add(q.id);
    visited.add(q.id);
    while (!queue.isEmpty()) {
      String cur = queue.poll();
      c.add(cur);
      for (Neighbor n : vectorIndex.neighbors(cur, minScore))
          {
        if (visited.add(n.id)) queue.add(n.id);
      }
    }
    clusters.add(c);
  }
  return clusters;
}
```

图 4-20: 会话聚类关键代码

```
# 任务
你是文本总结专家。给定一组语义相近的query，提炼共性并生成主题
    信息。

# 输入
句子用两个竖线分隔:
<sentences>
{q1 || q2 || q3 ...}
</sentences>

# 输出要求
1) summary: 30-50字主题摘要，直接给结论，不以“这些句子...”
    开头
2) keyword: 5-7个关键词，避免过于宽泛的词，需加具体场景限定
3) represent: 1-15条代表性query
4) 仅输出JSON，不要输出任何解释

# 输出(JSON)
{"summary":"...","keyword":["kw1","kw2"],"represent":["q1","
    q2"]}
```

图 4-21: 聚类主题总结提示词模板

答能力，并在输出侧返回可映射的引用；内容治理模块将重复、歧义与覆盖问题线上化为治理任务并与发布链路打通，形成可闭环的质量改进机制；数据洞察模块通过统一事件采集与指标查询支撑效果评估与问题定位，为后续实验评估与治理收益分析提供数据基础。

第五章　内容管理、应用与治理平台测试

本章参考师兄论文第五章的组织方式，从测试环境、单元测试与功能测试三个角度，对内容管理、应用与治理平台进行测试说明。测试目标包括：验证平台核心功能满足第三章需求分析中定义的业务用例；验证关键技术链路（发布口径、权限过滤、事件驱动索引、RAG 问答与治理闭环）在典型场景下能够稳定运行；并通过单元测试降低回归成本，提升迭代效率。

5.1　系统测试环境

平台在实现上由内容管理、内容应用、内容治理与数据洞察等模块构成，依赖关系型数据库与文档数据库存储内容资产，并在内容发布后构建全文索引与向量索引以支撑检索与智能问答。由于论文工程无法直接提供企业内部真实部署细节，本节从“部署形态 + 关键依赖组件 + 测试工具链”三个维度描述测试环境口径，便于复现测试结论。平台测试环境描述如表 **??**所示。

5.2　单元测试

单元测试用于在最小可控范围内验证模块内部逻辑正确性，并通过 Mock 隔离外部依赖以降低测试不稳定因素。平台的单元测试重点覆盖三类逻辑：第一类是权限裁决与状态机约束（例如 RBAC 与 content_auth 匹配、审批状态流转）；第二类是内容处理与索引构建的可重复性（例如切片构建规则、slice_id 稳定生成）；第三类是治理与洞察的关键算法单元（例如 TopK 候选生成、任务幂等 upsert 与复检回收）。

单元测试的基本原则如下：（1）边界清晰。对外部服务（数据库、索引服务、消息队列与大模型服务）通过 Mock 替身模拟输入输出，仅验证当前模块的业务逻辑与异常处理。（2）可重复执行。对时间、随机数与外部依赖进

表 5-1: 测试环境描述表

项目	说明
部署形态	服务以容器化方式部署于 Linux 环境，按模块拆分并独立扩缩容。
开发环境	开发与调试环境为 macOS 或 Linux，使用浏览器与接口调试工具完成联调。
数据存储	关系型数据库用于内容元数据、权限与任务数据；文档数据库用于案例详情等灵活结构字段。
检索与向量索引	全文检索索引用于关键词召回与高亮展示；向量索引用于语义召回与长尾覆盖，二者共同支撑 RAG。
消息通知与异步链路	内容发布后通过消息队列触发索引增量更新与数据回收等异步任务。
外部能力	智能问答与治理判别依赖大模型服务，图文与图片类检索依赖多模态 embedding 能力。
测试工具	前端功能测试使用 Chrome；接口与链路测试使用 Postman 或同类 HTTP 工具；必要时结合日志与 trace_id 进行链路定位。

行固定化处理，避免测试随运行环境变化而不稳定。（3）覆盖关键分支。对成功路径、非法状态、权限拒绝、幂等重试与异常退化等关键分支分别构造用例，降低线上故障风险。

以 content_auth 的“动作优先”裁决为例，单元测试需要覆盖不同 auth_tactic 与 auth_code 组合下的匹配结果，并验证“命中任一规则即放行、无规则命中则拒绝”的裁决逻辑。单元测试用例示例如图 **??**所示。

5.3 功能测试

功能测试用于验证平台对外可见能力与关键链路在真实交互下满足预期。本节基于第三章的核心用例集合，对内容管理、内容应用、内容治理与数据洞察模块设计功能测试用例。测试用例采用“步骤-预期结果”形式描述，覆盖正常流程、非法状态、权限约束、异步一致性与退化策略等场景。

```
def "test_content_auth_action_first"() {
  given: "构造访问者属性与授权规则集合"
  def visitor = new Visitor(isInternal: true, employeeId:
     1001, deptPath: ["d1","d2"])
  def rules = [
    new AuthRule(authTactic: "department", authVariable: "d2
       ", authCode: "r"),
    new AuthRule(authTactic: "employee", authVariable:
       "1002", authCode: "w")
  ]
  authDao.listRules(contentId) >> rules

  expect: "读动作命中部门规则放行，写动作不命中则拒绝"
  service.checkContentAuth(contentId, "r", visitor) == true
  service.checkContentAuth(contentId, "w", visitor) == false
}
```

图 5-1: 单元测试用例示例

5.3.1 内容管理功能测试

创建空间与应用是平台组织与权限的起点，用例用于验证空间隔离与应用边界能够正确生效。创建空间/应用的测试用例如表 **??**所示。

表 5-2: 创建空间/应用测试用例

用例编号	TC01
测试内容	创建空间与应用
测试步骤	1) 进入空间管理创建空间并填写负责人。2) 在空间内创建应用并选择支持的内容形态与组织方式。3) 进入成员与角色页面绑定默认角色。
预期结果	1) 空间与应用创建成功并可查询详情。2) 默认角色初始化完成且创建人具备对应权限。3) 不同空间的数据互相隔离。

内容生产链路覆盖创建、编辑、提审、审批与发布等过程，且要求读链路对外仅展示已发布内容。内容发布审批的测试用例如表 **??**所示。

跨应用引用用于复用权威内容并控制可见范围，用例用于验证“仅可引用已发布内容”与“引用幂等”约束。应用引用内容测试用例如表 **??**所示。

表 5-3: 内容提审与审批发布测试用例

用例编号	TC02
测试内容	内容提审、审批通过/驳回与发布口径
测试步骤	1) 创建一条 knowledge 内容并保存草稿。2) 发起提审进入待审批状态。3) 审批人执行驳回并填写意见。4) 再次提审并审批通过，内容进入已发布状态。5) 在门户与检索侧分别访问该内容。
预期结果	1) 草稿状态不对外可见。2) 驳回后内容回到可编辑状态并保留意见。3) 审批通过后内容对外可见，门户与检索均仅返回已发布口径。

表 5-4: 应用引用内容测试用例

用例编号	TC03
测试内容	应用引用已发布内容
测试步骤	1) 在源应用发布一条内容。2) 在目标应用发起引用该内容。3) 重复执行引用操作验证幂等。4) 在目标应用的列表与检索中查看引用内容。
预期结果	1) 未发布内容禁止被引用，已发布内容可引用。2) 重复引用不产生重复记录。3) 目标应用仅可见其引用集合内的内容且权限过滤生效。

5.3.2 内容应用功能测试

内容检索用于验证多字段召回、高亮与过滤逻辑是否正确。内容检索测试用例如表**??**所示。

表 5-5: 内容检索测试用例

用例编号	TC04
测试内容	内容检索与高亮展示
测试步骤	1) 在应用内准备多条不同类型内容并发布。2) 使用关键词分别命中 title、summary 与正文切片。3) 设置类目/标签/类型过滤并分页查询。4) 对不同权限用户重复检索。
预期结果	1) 关键词命中不同字段均可召回且排序合理。2) 返回结果包含高亮片段与摘要。3) 结构化过滤与分页生效。4) 越权内容不会出现在检索结果中。

智能问答用于验证“问题改写 + 多通道召回 + 重排 + 引用输出”的端到端链路，以及未命中与超时场景下的退化策略。智能问答测试用例如表 **??**所示。

表 5-6: 智能问答测试用例

用例编号	TC05
测试内容	RAG 智能问答端到端链路
测试步骤	1) 输入口语化问题触发问题改写。2) 观察 ES 与向量两路召回候选数量与 MinScore 过滤。3) 检查重排后 TopN 证据集合与 content 聚合情况。4) 生成回答并检查引用映射到 content 与 slice。5) 构造未命中与大模型超时场景。
预期结果	1) 改写后召回稳定性提升且不改变问题语义。2) 证据集合与问题强相关并包含可追溯引用。3) 未命中返回补充建议并进入覆盖度候选。4) 超时退化为检索摘要返回，且权限过滤始终生效。

5.3.3 内容治理与数据洞察功能测试

重复/歧义治理用于验证候选召回、LLM 判别与任务生成的闭环过程。重复/歧义治理测试用例如表 **??**所示。

表 5-7: 重复/歧义治理测试用例

用例编号	TC06
测试内容	重复/歧义检测与子任务生成
测试步骤	1) 在空间内准备两条高度相似内容并发布。2) 创建治理主任务并触发重复/歧义检测计算。3) 查看生成的重复/歧义子任务与内容对。4) 修改其中一条内容并重新发布，等待日巡检复检。
预期结果	1) 候选召回能够命中相似内容对并生成子任务。2) 子任务幂等写入，不重复生成。3) 内容修订后复检能够关闭已消除问题的子任务。

覆盖度治理用于验证“洞察聚类筛选薄弱主题 → 生成覆盖任务 → 补充内容 → 复检回收”的闭环。覆盖度治理测试用例如表 **??**所示。

数据洞察用于验证事件规范、trace_id 链路追踪与下钻能力。数据洞察测试用例如表 **??**所示。

表 5-8: 覆盖度治理测试用例

用例编号	TC07
测试内容	覆盖度任务生成与复检回收
测试步骤	1) 在问答链路制造未命中问题并沉淀为会话数据。2) 在洞察侧聚类生成低解决率主题。3) 在治理侧创建主任务并生成覆盖子任务。4) 补充相关内容并发布。5) 等待巡检复检回收。
预期结果	1) 覆盖任务能够以 cluster_id 或代表性 query 定位问题主题。2) 内容补充发布后复检能够将覆盖任务自动关闭。3) 任务状态与处理人记录完整可追溯。

表 5-9: 数据洞察链路测试用例

用例编号	TC08
测试内容	trace_id 链路追踪与会话下钻
测试步骤	1) 发起一次问答请求并记录 trace_id。2) 校验 rag_recall 与 llm_generate 等事件明细携带相同 trace_id 与 session_id。3) 在洞察侧按聚类主题下钻到会话明细。4) 按 trace_id 还原证据集合与引用内容。
预期结果	1) 关键事件均被采集且字段口径一致。2) 可按 trace_id 复现单次问答的召回与生成链路。3) 聚类下钻可定位高频主题与薄弱点，为治理任务生成提供输入。

5.4 本章小结

本章对内容管理、应用与治理平台的测试环境、单元测试与功能测试进行了说明。测试结果表明：平台在组织与权限边界、内容发布口径、事件驱动索引更新、RAG 问答引用可追溯、以及治理任务线上化闭环等关键链路上能够满足预期；数据洞察侧通过统一事件模型与 trace_id 关联实现链路还原与主题聚类下钻，为覆盖度治理与持续优化提供了数据基础。

第六章　总结与展望

6.1　总结

随着广告与商业化业务的快速发展，业务知识与运营规范等专业内容不断沉淀并快速演进，内容资产规模增长带来了内容分散、复用困难、维护成本高与质量不可控等问题。围绕上述问题，本文面向广告与商业化场景设计并实现了一套内容管理、应用与治理平台，以内容全生命周期为主线，对内容建模、组织、应用与治理进行系统化建设，从而提升内容资产的统一管理能力、应用效果与质量可控性。

本文工作的主要贡献可概括为以下四个方面。（1）构建了基于空间与应用分层的内容组织模型，并实现多形态内容的统一管理能力。平台将空间作为业务隔离边界，将应用作为消费场景与配置边界，支持文章、问答、课程与案例等多种内容形态的规范化管理；通过类目树与标签体系完成内容组织，通过跨应用引用机制实现权威内容复用，并以审批发布流程约束对外口径，保证内容在不同消费入口下的一致性。（2）实现了面向内容应用的检索与智能问答能力，并以发布口径与权限过滤贯穿读链路。平台在页面化展示与内容检索的基础上，引入 RAG 智能问答能力：通过内容切片、全文索引与向量索引的双通道检索，提高召回的准确性与覆盖率；通过问题改写、多通道召回、重排与证据组织生成高相关证据集合，并在输出侧返回可映射引用，降低幻觉风险并支持后续治理。（3）构建了以重复度、歧义度与覆盖度为核心的内容治理机制，并实现治理任务线上化闭环。平台结合向量检索与大语言模型完成对重复与歧义问题的自动检测与结构化判别，并将结果转化为可流转的治理任务；覆盖度治理复用洞察侧会话聚类结果定位薄弱主题并生成覆盖任务，结合内容补充、发布与复检回收形成闭环机制，使治理从一次性离线分析转化为可持续运营。（4）实现了数据洞察能力为应用与治理提供数据支撑。平台定义统一事件模型与关键链路埋点规范，通过 session_id 与 trace_id 实现端到端追踪与明细还原；洞察侧提供会话聚类下钻与指标查询能力，用于定位高频问题与薄弱点，并为覆盖度治理与策略优化提供依据。

通过第五章的单元测试与功能测试验证，平台的组织与权限边界、审批发布口径、事件驱动索引更新、RAG 问答引用可追溯、治理任务线上化与复检回收等关键链路能够满足预期，为广告与商业化场景下的内容资产管理与应用提供了可落地的工程方案。

6.2 展望

虽然本文实现的平台能够覆盖内容管理、应用与治理的核心需求，但仍有进一步优化空间，主要体现在以下几个方面。（1）内容形态与结构化表达仍需增强。当前平台已支持文章、问答、课程与案例四类基础内容形态，并在切片阶段对图片与表格提供了特殊处理策略，但对更复杂的结构化知识（如规则、流程、指标定义与参数表等）的建模与检索仍有提升空间。后续可引入更细粒度的结构化抽取与 Schema 约束，使内容既可被页面化展示，也可被更高质量地召回与引用。（2）索引与检索链路的成本与时延仍需优化。平台采用事件驱动的增量索引构建机制，但在内容规模进一步增大时仍需更精细的分批、优先级与资源隔离策略；在 RAG 链路上也可通过召回策略自适应、缓存与近似搜索等手段降低时延，并在保证可解释性的前提下优化重排效果。（3）治理自动化与可操作性仍需提升。重复与歧义治理目前以任务化分发与人工处理为主，后续可探索“建议式治理”，例如自动给出合并候选、下线建议与引用调整方案，并将治理建议与审批发布流程更紧密地联动，以降低人工成本；覆盖度治理可进一步与知识体系驱动路径结合，使覆盖分析既反映真实用户需求，也能对齐业务知识体系结构。（4）安全合规与隐私保护需要持续加强。平台在权限层采用 RBAC 与 content_auth 叠加裁决，后续可进一步支持更通用的属性基访问控制与审计追踪能力；在数据洞察场景需持续完善最小化采集、脱敏与明细访问控制，确保数据使用满足合规要求。

后续工作将围绕上述方向持续迭代平台能力，进一步提升内容资产的复用效率、问答体验与治理闭环效果，为广告与商业化业务提供更稳定、可控与可持续演进的内容基础设施。

参考文献

附录 A　附录示例

A.1　main 函数

```
int main()
{
        return 0;
}
```

致　　谢

致谢示例

版权及论文原创性说明

任何收存和保管本论文的单位和个人，未经作者本人授权，不得将本论文转借他人并复印、抄录、拍照或以任何方式传播，否则，引起有碍作者著作权益的问题，将可能承担法律责任。

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写的作品成果。本文所引用的重要文献，均已在文中以明确方式标明。本声明的法律结果由本人承担。

作者签名：_____

_____年_____月_____日

《学位论文出版授权书》

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：_____

_____年_____月_____日

论文题名	面向广告与商业化内容的管理、应用与治理平台的设计与实现				
研究生学号	522024320076	所在院系	软件学院	学位年度	2026
论文级别	☐ 学术学位硕士　☐ 专业学位硕士 ☐ 学术学位博士　☐ 专业学位博士 （请在方框内画钩）				
作者 Email	522024320076@smail.nju.edu.cn				
导师姓名	刘峰 高级工程师				

论文涉密情况：

☐ 不保密

☐ 保密，保密期 (_____年_____月_____日 至 _____年_____月_____日)

注：请将该授权书填写后装订在学位论文最后一页（南大封面）。